A Problem-Driven Design Framework for Physics-Informed Neural Networks

Rashmi R[1*], Siya Patil[1], Ruchika Todi[1], Smriti Prashant Vasisht[1], Vipul Reddy Gundalu[1], Mallanagouda Patil[2]

[1*]Department of Information Science and Engineering, B.M.S College of Engineering, Bengaluru, India .
[2]Department of Computer Science Engineering, R.V Institute of Technology and Management, Bengaluru, India .

*Corresponding author(s). E-mail(s): rashmir.ise@bmsce.ac.in;
Contributing authors: siyapatil.is23@bmsce.ac.in; ruchikatodi.is23@bmsce.ac.in; smriti.is23@bmsce.ac.in; vipulreddy.is24@bmsce.ac.in; mallanagoudap.rvitm@rvei.edu.in;

Abstract

Physics-informed neural networks (PINNs) offer a broad range of choices in formulation, constraint enforcement, network architecture, derivative and operator evaluation, and training. The central practical challenge is therefore not the availability of PINN variants, but selecting an appropriate configuration for a given problem. This survey addresses this challenge by treating PINN method selection as an evidence-synthesis problem. The design space is organized into three layers with distinct functional roles: a physics layer containing formulation and closely coupled constraint enforcement, a representation layer containing network architecture, and a computational-realization layer containing derivative and operator evaluation. These choices are mapped to mathematical, geometric, data, and computational characteristics of the target problem, and comparative evidence is assessed according to its strength, comparability, and independent corroboration. Across the retained comparative corpus, the evidence does not support a universal PINN configuration. Instead, several choices admit conditional recommendations: constrained enforcement is strongly supported when exact boundary or initial satisfaction is required and an admissible construction exists, while formulation, architecture, optimization, sampling, and operator evaluation remain dependent on problem and computational characteristics. Numerical alternatives to automatic differentiation are repeatedly reported as

competitive in accuracy and training cost in the tested settings, but no universal numerical realization is established. These findings motivate a problem-driven decision procedure that accounts for structural admissibility, comparative evidence, and documented interactions between design choices. The survey also examines emerging directions including KAN/PIKAN, attention-based models, operator learning, automated design, data-assisted and uncertainty-aware PINNs, scalable and distributed approaches, and quantum or hybrid quantum-classical PINNs, distinguishing demonstrated feasibility from evidence sufficient to support general design recommendations.

Keywords: physics-informed neural networks, scientific machine learning, method selection, evidence synthesis, design space, taxonomy, quantum machine learning, benchmarking

1 Introduction

A physics-informed neural network (PINN) is a neural network trained to solve differential equations by combining a governing PDE or ODE with boundary and initial conditions and, where available, observational data. The network approximates the solution $u(x, t)$, automatic differentiation or another operator-evaluation scheme computes the required derivatives, and the resulting equation residuals and constraints are incorporated into a training objective [1]. This allows PINNs to combine physical models with sparse or noisy observations and to address forward, inverse, and data-assimilation problems within a common framework.

Since their formalization in 2019, PINNs have been applied across fluid mechanics [2–4], heat transfer [5], solid and fracture mechanics [6–8], biomedical modeling [9], geophysics [10], and photonics [11], with broader reviews available elsewhere [12, 13]. This breadth has produced a large range of PINN formulations, constraint-enforcement strategies, architectures, operator-evaluation schemes, and training methods.

These variants address different design questions and cannot be treated as a flat catalog. Formulation and constraint enforcement determine the mathematical conditions imposed during training, architecture determines how the solution is represented, and computational realization determines how the required derivatives or operators are evaluated. Their suitability also depends on characteristics of the target problem, including its mathematical structure, geometry, available data, and computational constraints. Auxiliary choices such as sampling, loss weighting, and optimization can further affect these decisions. An overview of these roles is shown in Figure 1.

Existing reviews provide taxonomies, application surveys, bibliometric analyses, and focused discussions of particular PINN components [12–16]. More recent benchmarking and review work has also begun to relate method performance to problem properties [17, 18]. However, the literature retained in this survey does not provide a common framework that simultaneously organizes principal design choices by functional role, describes relevant problem characteristics using a common vocabulary,

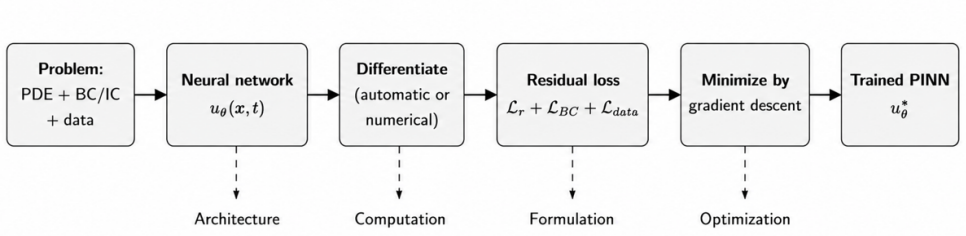


Fig. 1 Workflow of a physics-informed neural network. Architecture defines the network, formulation specifies the residuals in the loss, and computation determines how derivatives are obtained.

synthesizes comparative evidence, accounts for interactions between choices, and converts these observations into an explicit design procedure. This survey addresses that gap with a problem-driven design framework for PINNs.

Contributions of This Survey

This survey brings these elements together by:

1. **Organizing the PINN design space.** Formulation and constraint enforcement are treated as a physics layer, architecture as a representation layer, and computational realization as an operator and derivative-evaluation layer.
2. **Mapping problem characteristics to design choices.** Mathematical, geometric, data, and computational properties of a problem are related to the design choices they constrain or motivate.
3. **Synthesizing and grading comparative evidence.** Comparative findings are assessed according to their strength and independent corroboration, distinguishing evidence strength from methodological quality or publication prominence.
4. **Accounting for interactions between design choices.** The survey identifies interactions that affect how individual design choices should be interpreted and selected.
5. **Assessing emerging directions.** KAN/PIKAN, attention-based models, operator learning, adaptive and automated design, data-assisted and uncertainty-aware PINNs, scalable approaches, and quantum or hybrid quantum-classical PINNs are examined using the same comparative perspective.

The practical outcome is a problem-driven design procedure (Section 7) that guides the selection of PINN design choices from problem characteristics and the available comparative evidence.

2 Methodology

2.1 Research questions

This survey is organized around five research questions. **RQ1.** What are the principal design dimensions of a PINN, and how should they be structured relative to one another (Section 3)? **RQ2.** Which mathematical, geometric, computational, and data

characteristics of a problem are associated with particular design choices (Section 4)? **RQ3.** What comparative evidence exists for competing design choices at each layer of the design space (Section 5)? **RQ4.** How comparable and reproducible are the published comparisons, and what methodological limitations affect their interpretation (Section 5.2)? **RQ5.** Which methodological directions in this space are genuinely emerging, and how mature is the evidence supporting them (Section 8)? Together, RQ1–RQ4 support the problem-driven design procedure of Section 7, while RQ5 extends the same perspective to emerging directions.

2.2 Search strategy and evidence-synthesis scope

This survey uses a structured, curated evidence synthesis to develop a problem-driven design framework from heterogeneous comparative evidence. The corpus was assembled through targeted literature searches, citation tracing, and relevance-based screening, with emphasis on studies that compare PINN design choices. The synthesis is intended to organize and interpret comparative evidence rather than estimate method prevalence or produce pooled effect sizes. Attention is therefore given to experimental comparability, independence of findings, and the strength of support for each design recommendation.

Existing systematic reviews and broad PINN surveys provide taxonomies, trend analyses, application coverage, and methodological overviews [15, 16, 19, 20]. This survey addresses a narrower question: given a problem, which PINN design choices are supported by comparative evidence, under what conditions, and with what evidential strength? It therefore complements existing reviews by connecting problem characteristics to design choices while accounting for interactions and cross-study comparability.

The literature was identified through targeted searches of arXiv (cs.LG, math.NA, and physics), Google Scholar, and the Semantic Scholar academic graph, supplemented by backward and forward citation tracing from general PINN surveys in Section 1 and retained comparative studies. Search terms combined "physics-informed neural network" or "PINN" with terms covering the design dimensions in Section 3 and problem characteristics in Section 4, including formulation, constraint enforcement, operator evaluation, architecture, optimization, sampling, loss weighting, domain decomposition, operator learning, quantum methods, stiffness, discontinuities, shocks, high frequency, multiscale structure, complex geometry, high dimensionality, inverse problems, data assimilation, and sparse or noisy data.

Treatment of preprints. Preprints were used for discovery and recent or emerging work; peer-reviewed versions were cited where available, and preprint/published pairs were not counted separately. Preprints without a peer-reviewed version do not support evidence grades above E3.

2.3 Eligibility, screening, and data extraction

A source was included in the comparative synthesis only if it compared at least two design choices, or provided a controlled ablation of at least one, on a stated problem.

General method-introduction papers reporting only a single method without a controlled baseline are cited for method existence and mechanics (Sections 3 and 8) but not as comparative evidence. Studies comparing a PINN only with a classical numerical solver or purely data-driven model are likewise treated as applications, unless the physics-informed-versus-data-driven contrast is itself the design choice under test (Section 4.2).

For each retained study, the survey recorded the design choices compared, problem characteristics, reported outcomes, and relevant methodological details where available. Unreported values are recorded as "not reported" (NR) rather than inferred. The retained comparative studies provide the basis for the evidence synthesized throughout Section 5, Section 6, and Section 8.

2.4 Comparative-evidence and grading criteria

Comparative sources were assigned the five evidence grades defined in Section 5.1. The grades characterize the strength and corroboration of a comparative finding, rather than the general quality of an individual study. Independent corroboration was assessed with attention to research-group affiliation so that repeated findings from the same group were not counted as independent replication. Citation count, venue prestige, and recency do not affect the evidence grade.

The grading characterizes the evidence available within the survey's retained literature rather than providing an exhaustive assessment of all PINN literature. Accordingly, research gaps identified in Section 9 are restricted to gaps supported by the surveyed evidence rather than inferred from the absence of an identified source.

3 The PINN Design Space

Figure 2 summarizes the design space developed in this section as three coupled layers. Formulation (Section 3.1) specifies the mathematical objective, while constraint enforcement (Section 3.2) is tightly coupled to formulation because the available enforcement mechanisms depend on the formulation chosen. Architecture (Section 3.3) specifies the neural backbone and can be paired with different formulation and enforcement mechanisms. Operator and derivative evaluation (Section 3.4) specifies how the formulation's required derivatives are computed and is the most loosely coupled layer. We refer to these as the physics, representation, and computational-realization layers, respectively. They have different degrees of coupling and documented interactions (Section 6). Auxiliary mechanisms (Section 3.5) enhance these layers rather than constituting a separate layer, while Section 4 relates the design choices to problem characteristics.

3.1 Formulation

Formulation specifies how the governing equation is converted into a trainable objective and therefore what mathematical quantity the loss penalizes. The three principal formulations considered here are strong (residual), weak (variational/Galerkin), and

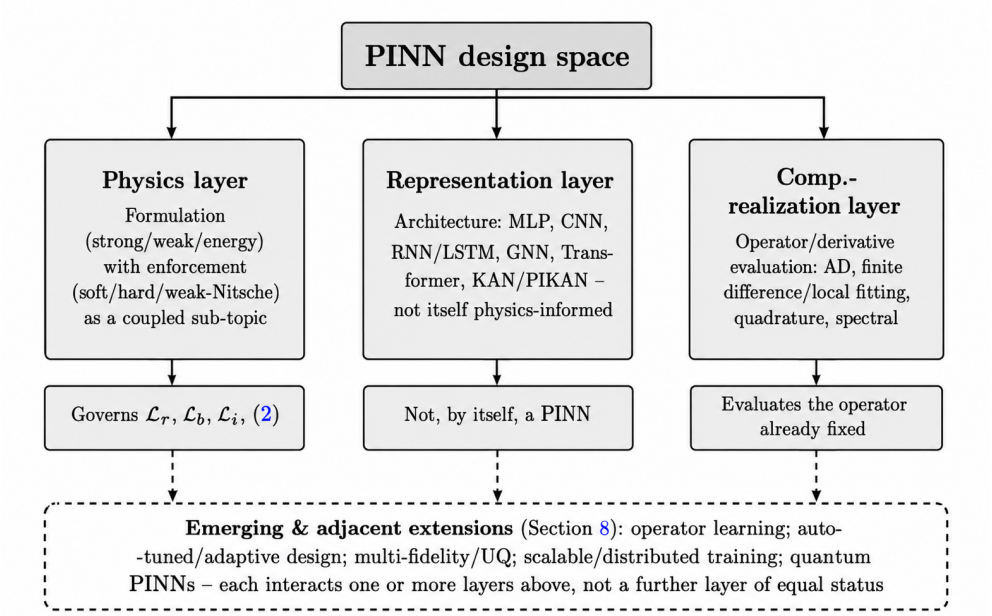


Fig. 2 The PINN design space organized into three layers: physics formulation and enforcement, network representation, and computational realization. Emerging and adjacent approaches such as operator learning, adaptive design, multi-fidelity learning, and quantum PINNs interact with one or more of these layers rather than forming a separate layer.

energy-based formulations. The choice affects not only what is learned but also which boundary-condition enforcement mechanisms are available.

Given a governing equation and boundary/initial condition on a domain Ω with boundary $\partial\Omega$,

$$\mathcal{N}[u;\lambda](x,t) = 0 \text{ on } \Omega, \qquad \mathcal{B}[u](x,t) = 0 \text{ on } \partial\Omega \text{ (and } t = 0), \tag{1}$$

as expressed in Equation (1), a PINN approximates u by a network u_θ trained on a composite objective

$$\mathcal{L}(\theta) = \lambda_r \mathcal{L}_r(\theta) + \lambda_b \mathcal{L}_b(\theta) + \lambda_i \mathcal{L}_i(\theta) + \lambda_d \mathcal{L}_d(\theta), \tag{2}$$

given by Equation (2), where $\mathcal{L}_r$, $\mathcal{L}_b$, $\mathcal{L}_i$, and $\mathcal{L}_d$ penalize the governing-equation residual, boundary condition, initial condition, and any available data, respectively, and $\lambda_r, \lambda_b, \lambda_i, \lambda_d$ are loss weights (Section 3.5). Formulation, taken up next, fixes what $\mathcal{L}_r$ means; enforcement (Section 3.2) fixes how $\mathcal{L}_b$ and $\mathcal{L}_i$ enter (2), since hard enforcement removes the corresponding conditions from the loss by construction rather than merely reweighting them. Three principal formulations appear in the literature.

In the **strong (residual) form**, the operator is evaluated pointwise at N_r collocation points and the loss penalizes the squared residual directly,

$$\mathcal{L}_r(\theta) = \frac{1}{N_r} \sum_{k=1}^{N_r} \left| \mathcal{N}[u_\theta; \lambda](x_k, t_k) \right|^2, \tag{3}$$

as defined in Equation (3), as in the original PINN [1]. This is the standard formulation and requires differentiability of the network approximation up to the order of the differential operator.

In the **weak (variational/Galerkin) form**, the residual is projected against a set of test functions $\{v_k\}$ and integrated,

$$\mathcal{L}_r(\theta) = \sum_k \left| \int_\Omega \mathcal{N}[u_\theta; \lambda](x, t)\, v_k(x, t)\, dx\, dt \right|^2, \tag{4}$$

as given by Equation (4), where integration by parts can reduce the derivative order required of u_θ, at the cost of an additional quadrature step (Section 3.4), as in variational PINNs and their *hp*-refined extensions [21].

In the **energy (variational-principle) form**, where the governing equation admits a variational principle which is an energy functional $E[\cdot]$ whose Euler–Lagrange equation is $\mathcal{N}[u; \lambda] = 0$ and the loss minimizes that functional directly,

$$\mathcal{L}_r(\theta) = E[u_\theta], \tag{5}$$

as defined in Equation (5), as in the Deep Ritz method [22]. We treat this as distinct from the weak residual formulation because the objective is constructed from an energy or variational functional rather than from direct minimization of a pointwise PDE residual. Energy-based formulations can also reduce the derivative order required of the network approximation relative to strong-form residual minimization [23, 24].

A phase-field fracture study using a variational-energy PINN formulation reported this lower derivative-order requirement and used an ansatz-based treatment of boundary conditions [23]. A finite-strain hyperelasticity study likewise identified the lower derivative order of the Deep Energy Method as an advantage over strong-form PINNs, while showing that both approaches can struggle with localized stress and displacement concentration features. Its mixed Deep Energy Method introduced stress as an additional network output to address these features [24]. Together, these studies support lower differentiability requirements and natural compatibility with variational treatments of boundary conditions, while showing that the formulation does not by itself resolve localized solution features.

Formulation is a statement about what mathematical objective is optimized, whereas enforcement addresses how boundary and initial data are imposed on that objective. In particular, the *weak (variational/Galerkin) formulation* introduced above is an integral residual obtained by multiplying against a test function. It is distinct from *weak (Nitsche-type) enforcement*, introduced in Section 3.2, which is one way of imposing boundary or initial conditions. The two uses of "weak" therefore refer to

Table 1 Principal boundary and initial-condition enforcement mechanisms.

Mechanism	Imposition	Main advantage	Main limitation
Soft	Weighted penalty terms in the loss	Flexible and broadly applicable	Requires appropriate loss weighting
Hard	Network output constrained by construction	Exact satisfaction	Requires an admissible construction; geometry dependent
Weak/Nitsche	Boundary terms incorporated into the weak residual	Compatible with variational formulations	Requires a suitable weak formulation

different concepts. Thus, once formulation is chosen, the next design decision is how that formulation is applied to boundary and initial conditions; this choice is available only for certain formulations and is therefore not independent.

3.2 Boundary and initial-condition enforcement

Enforcement specifies how boundary and initial conditions are incorporated into training. Conditions can be enforced softly as weighted penalty terms, hard by constraining the network output by construction, or through variational methods that modify the weak-form residual. The available mechanisms depend on the formulation already chosen, so enforcement and formulation are treated as tightly coupled design choices: formulation determines *what* the loss penalizes, while enforcement determines *how* boundary and initial data enter that loss. The principal mechanisms are summarized in Table 1, while their comparative performance and appropriate use are discussed below.

Soft (penalty) enforcement adds conditions to the loss as weighted penalty terms [1, 25]. This is the default approach and applies to different operators and boundary types, but introduces additional loss terms whose relative weights can affect training.

Hard (construction) enforcement transforms the network output so that a condition holds by construction, typically as

$$u_\theta(x) = g(x) + d(x)\, N_\theta(x), \tag{6}$$

as expressed in Equation (6), where g satisfies the boundary/initial condition, d vanishes exactly on $\partial\Omega$ (and, for initial conditions, at $t = 0$), and N_θ is an otherwise unconstrained network. Such constructions include trial-solution ansatzes [26], approximate distance functions [27], and constrained or hybrid formulations that combine exact constraints with penalty or augmented-Lagrangian terms for inverse problems [28]. When hard enforcement is used, $\mathcal{L}_b$ and/or $\mathcal{L}_i$ is removed from (2), since the corresponding condition already holds for every admissible θ.

Weak (variational, Nitsche-type) enforcement imposes conditions by modifying the weak-form residual itself, following the principle of Nitsche's method. A controlled comparison of four enforcement methods across collocation and variational

PINNs, using a fixed architecture and optimizer schedule over five problem classes, provides a direct comparison of these alternatives [29].

Comparative evidence. The available evidence does not support a universal ranking of the three mechanisms. In the controlled comparison of [29], normalized distance-function hard enforcement gives the strongest overall results among the tested methods, while an unnormalized version can produce suboptimal or non-convergent results. Soft enforcement remains effective when its weight is appropriately chosen, and weak/Nitsche enforcement is competitive with a well-tuned soft penalty in some cases. The same study reports comparable performance across the more regular elliptic test case, while soft enforcement is marginally more accurate than hard enforcement for one convection problem under the variational formulation. A unified theoretical error analysis likewise finds weaker error estimates when boundary and initial conditions are imposed through penalties than when they are imposed exactly [30].

Independent studies report benefits of constrained enforcement in several other settings. A hard-constraint projection for heterogeneous subsurface flow reports higher accuracy, lower data requirements, and greater robustness to observation noise than soft-constraint models [31]. An augmented-Lagrangian formulation reports substantial accuracy improvements over soft-penalty PINNs for forward and inverse problems [32]. In an advection–diffusion problem, a hard-constraint output transformation combined with a multi-scale subnetwork and Fourier-induced activations outperforms both a standard PINN and the corresponding soft-enforcement model [33]. For stiff, time-dependent problems, exact embedding of the *initial* condition, rather than exact boundary enforcement, is reported to stabilize training; neural-tangent-kernel analysis attributes this to an implicit time-marching effect and reduced spectral bias [34].

These results show that constrained enforcement can affect not only boundary satisfaction but also accuracy, data efficiency, robustness, and training stability. However, the preference for exact enforcement remains conditional. In an acoustic-wave problem with a continuous time-dependent point source, fully soft initial and boundary conditions produced the lowest prediction error among four combinations of soft and hard treatment [35]. Together with the convection result of [29], this provides independent evidence that a well-tuned soft penalty can be preferable in some settings.

Therefore, hard enforcement is a strong default when exact boundary or initial satisfaction is required and an admissible construction exists, but the evidence does not support "hard enforcement is better" as a universal rule. Its effectiveness depends on the problem, formulation, and particular construction used; soft enforcement remains competitive and can be preferable when its weighting is well chosen. Enforcement should therefore be selected jointly with formulation rather than treated as an independent optimization axis (Section 5.2).

The coupling is particularly evident for energy-based formulations. In phase-field fracture, the Deep Ritz formulation was reported to face difficulty with soft Dirichlet enforcement, leading to an explicitly boundary-satisfying ansatz across six fracture problems [7]. An independent fracture study likewise used exact boundary-condition imposition with an energy formulation, reporting advantages associated with more

robust enforcement and lower derivative order [23]. These results illustrate that enforcement can be especially important when coupled to a specific formulation.

Having established formulation and enforcement as the coupled physics layer, the next design decision is the neural architecture used to represent the solution.

3.3 Architecture: neural representation

Architecture is the neural network backbone used to approximate the solution. Unlike formulation and enforcement, which are physics decisions, architecture is a representation choice. A backbone becomes physics-informed only when trained with a formulation (Section 3.1) and associated enforcement mechanism (Section 3.2). The principal architecture families considered in this survey are summarized in Table 2.

The multilayer perceptron (MLP) is the default backbone in the great majority of PINN variants [1]. Convolutional architectures are suited to grid-structured or geometry-mapped domains, while recurrent and graph-based architectures target sequential/temporal and irregular/relational structure, respectively. Attention/Transformer and KAN/PIKAN architectures are discussed in greater detail in Sections 8.1 and 8.2, since their comparative evidence remains less mature.

DeepONet and Fourier neural operators (FNOs) are an adjacent case. Unlike conventional PINNs, unconstrained operator-learning models learn mappings between function spaces from data rather than enforcing a governing equation [36, 37]. They therefore lie outside the core formulation and enforcement axes. When a physics-informed residual is incorporated into their training, they enter the physics-informed setting as neural operators, including physics-informed DeepONet and PINO-type formulations [38]. These cases are discussed in Section 8.3.

Hybrid architectures provide another boundary case. A convolutional, finite-volume-informed hybrid replaces automatic differentiation with a local fitting operator [39]. Quantum-classical PINNs likewise combine quantum and classical components; in one documented case, the quantum component accounts for only about 0.1–0.3% of the model's total trainable parameters [40]. Quantum PINNs are therefore treated in Section 8.7 as intersecting the representation and computational-realization layers rather than as a separate principal dimension of the design space.

Architecture is not an isolated choice: its performance can depend on the optimizer, parameter budget, and other aspects of the training configuration. These interactions are examined in Section 6.

With architecture chosen, the final design decision concerns how the derivatives and operators specified by the formulation are numerically evaluated. This is the computational-realization layer: it changes how the required quantities are computed, not the mathematical quantities defined by the formulation.

3.4 Computational realization of operators and derivatives

Computational realization specifies how the derivatives and integrals required by a chosen formulation are evaluated numerically during training. The formulation determines *what* mathematical quantity must be evaluated, while computational realization determines *how* it is evaluated. The principal alternatives considered in this survey are

summarized in Table 2. For example, a strong-form PINN requires pointwise differential operators, whereas weak and energy formulations require numerical integration in addition to the corresponding differential quantities.

Automatic differentiation (AD) is the default in most PINN implementations. It evaluates derivatives of the neural network's computational graph to numerical precision, without introducing the truncation error associated with a finite-difference approximation. For a network $u_\theta(x)$, a first derivative can be written as

$$u_x(x) = \frac{\partial u_\theta(x)}{\partial x}. \tag{7}$$

This first derivative is given in Equation (7). Reverse-mode AD evaluates such derivatives through the computational graph in a manner closely related to backpropagation [41]. Its computational and memory cost can become substantial for high-dimensional or high-order operators, particularly when large derivative tensors must be formed. Several approaches therefore retain AD while changing how derivatives are evaluated. Separable constructions process inputs axis by axis and have been reported to reduce error and computational cost relative to a standard fully connected PINN across several PDE benchmarks [42]. Hutchinson trace estimation avoids explicitly forming the full Hessian when evaluating a second-order residual, instead using Hessian–vector products obtained through Taylor-mode AD; the same idea is extended to tensor–vector products for a fourth-order biharmonic operator in high-dimensional problems [43]. These approaches modify the cost or memory footprint of differentiation without replacing AD itself.

Numerical operator evaluation instead replaces automatic differentiation with an approximation of the required differential operator. For example, a first derivative can be approximated by the central finite-difference stencil

$$u_x(x_i) \approx \frac{u_\theta(x_i + h) - u_\theta(x_i - h)}{2h}. \tag{8}$$

Unlike AD, the resulting derivative in Equation (8) depends on the numerical discretization, including the stencil, point distribution, boundary treatment, and operator order. Several studies nevertheless report competitive or improved computational performance. Meshless radial-basis-function finite differences achieve comparable losses with two- to four-fold faster training on linear and nonlinear Poisson problems and remain applicable to point clouds on irregular geometries [44]. A finite-difference residual for Laplace's and Burgers' equations is likewise reported to have lower computational cost with comparable error to an automatic-differentiation residual [45]. Hybrid schemes can retain automatic differentiation near complex boundaries while using finite differences in the interior [46]. A finite-volume residual eliminates explicit derivative terms and enforces conservation over control volumes; on lid-driven cavity flow, including forward and inverse problems, it is reported to achieve higher accuracy than an automatic-differentiation PINN at roughly one tenth of the training time [47]. Radial-basis-function differential quadrature provides another numerical alternative

for high-order derivatives on scattered points [48]. A further study uses finite differences for an auxiliary residual-gradient term while retaining automatic differentiation for the governing residual, with six random seeds used in the comparison [49].

The evidence indicates that replacing AD is not necessarily only a computational trade-off. In some problems, the choice of operator evaluation also affects the quality of the resulting solution. A phase-field fracture study reports qualitatively incorrect solutions and stress-localization behaviour when automatic differentiation is used with a smooth-activation network, while finite-element shape-function gradients on a discretized domain produce faster and more accurate results [7]. The approach, however, reintroduces a mesh into an otherwise mesh-free formulation. Independently, a differential-quadrature study of steady incompressible flow reports that its numerical differentiation scheme can solve cases, particularly at higher Reynolds number, that an automatic-differentiation PINN struggles to solve, and also reports nonphysical solutions under sparse sampling [48]. These are documented problem-specific cases rather than evidence that AD is generally unreliable.

A convolutional, finite-volume-informed hybrid provides a further example in which operator evaluation is integrated with the network architecture. It replaces automatic differentiation with a local fitting operator while retaining the physics-informed training framework [39]. This illustrates that computational realization can interact with the representation layer rather than remaining entirely independent of architecture.

Quadrature is more tightly constrained by formulation than the numerical alternatives above. When the weak or energy formulation of Section 3.1 produces an integral, that integral must be evaluated numerically, for example through a weighted sum

$$\int_{\Omega} f(x)\,dx \approx \sum_{j=1}^{N_q} w_j f(x_j). \tag{9}$$

Thus, quadrature in Equation (9) is not an independent alternative to automatic differentiation in the same sense as a finite-difference stencil. Rather, the formulation determines whether integration is required, while the quadrature rule determines how it is evaluated. This is a direct manifestation of the coupling between formulation and computational realization (Section 6).

Specialized operator evaluation is used when a particular operator is difficult to evaluate efficiently through ordinary automatic differentiation. Fractional PINNs provide one documented example: integer- order terms continue to use automatic differentiation, while the fractional operator is evaluated using a numerical, basis-based discretization [50]. The computational realization is therefore specialized to the operator without requiring a separate PINN formulation.

Quantum parameter-shift evaluation provides the corresponding case for quantum-classical PINNs. For a parameterized quantum circuit, the derivative of an expectation value with respect to a circuit parameter can be evaluated using the parameter-shift rule,

$$\frac{\partial \langle O \rangle}{\partial \theta} = \frac{1}{2}\left[\langle O \rangle_{\theta+\pi/2} - \langle O \rangle_{\theta-\pi/2}\right]. \tag{10}$$

Quantum-classical PINNs can therefore use the parameter-shift evaluation in Equation (10) for quantum parameters while retaining automatic differentiation for their classical components [51]. As discussed in Section 8.7, this places quantum PINNs at the intersection of the representation and computational-realization layers rather than making quantum evaluation a separate principal layer of the design space.

Taken together, the evidence shows that automatic differentiation is the default computational realization, but not a universally optimal one. Numerical alternatives can reduce training cost, handle high-order or scattered-point operators more flexibly, and in some documented cases improve the reliability of the resulting solution. At the same time, some computational choices are imposed or strongly constrained by the formulation itself, as with quadrature in weak and energy formulations. The relevant design decision is therefore not simply whether to use automatic differentiation, but which evaluation strategy best matches the operator, formulation, sampling geometry, and computational requirements of the problem. The remaining mechanisms considered in the design space act primarily at the training level and are therefore treated as auxiliary design choices.

3.5 Auxiliary design mechanisms

A further set of mechanisms affects how effectively a chosen formulation, architecture, and computational-realization combination is trained, without replacing the choice made at any of the three principal layers. Four mechanisms recur throughout this survey: domain decomposition, collocation-point sampling, loss weighting, and optimizer choice.

Domain decomposition partitions the domain, and often the training computation, across sub-networks. Conservative and extended PINNs [52, 53], finite-basis PINNs [54], and parallelized variants [55] use different decomposition strategies, with generalization and scaling benefits reported for particular schemes [56, 57]. Its primary role is therefore to improve scalability and handle complex or large domains rather than to define a separate formulation or architecture.

Collocation-point sampling controls where the residual is evaluated during training. Residual-based adaptive refinement [58, 59], residual-and-gradient-based refinement [60], and point-moving strategies [61] increasingly target regions where additional resolution is most useful while addressing the computational cost of high-dimensional resampling.

Loss weighting rebalances the terms of (2). Approaches include gradient-flow-based, neural-tangent-kernel-based, and self-adaptive schemes [25, 62, 63]. A direct comparison of four weighting schemes across forward and inverse PDE problems reports an advantage for relative-loss balancing with random lookback in the tested settings [64]. However, a subsequent study shows that the ranking depends on the loss function and activation function [65]. Thus, loss weighting has useful comparative evidence but no universal optimum.

Optimizer choice and scheduling determines how the resulting objective is minimized. The conventional workflow commonly combines Adam with L-BFGS [58],

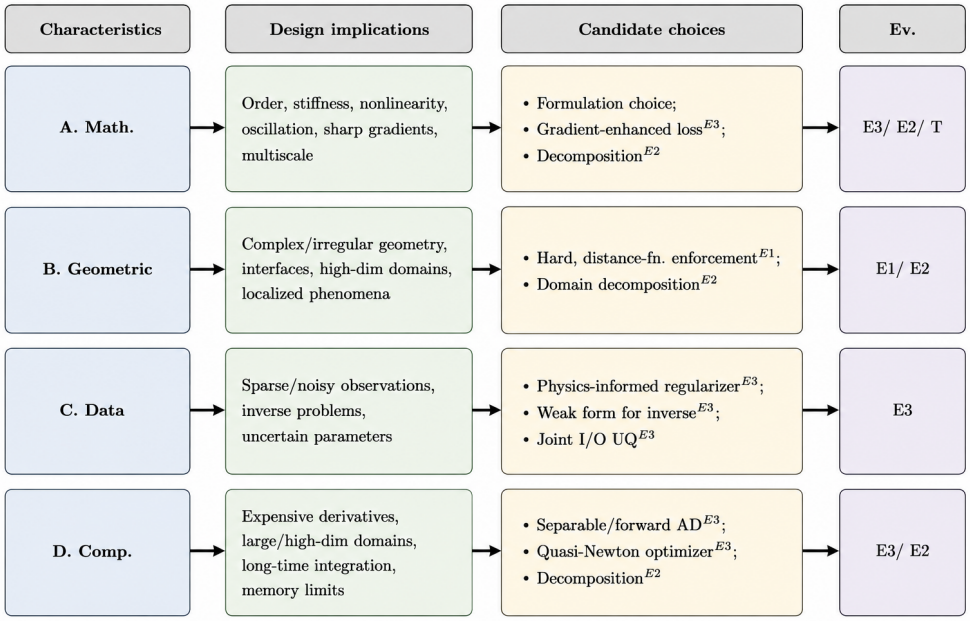


Fig. 3 Problem characteristics mapped to design implications, candidate PINN choices, and evidence grades. E1/E2 denote established or corroborated evidence, E3 single-study evidence, and T theoretical support.

while recent comparisons examine self-scaled quasi-Newton methods and curvature-aware approaches [66, 67]. These results indicate that optimization remains an active design choice whose effectiveness depends on the problem and computational budget.

Taken together, these mechanisms primarily control how a selected PINN configuration is trained rather than defining the configuration itself. Domain decomposition addresses scalability, adaptive sampling controls where computational effort is concentrated, loss weighting balances competing objectives, and optimization determines how the resulting objective is minimized. Their effects can depend on the formulation, architecture, loss, and other training choices (Section 6). They are therefore treated as auxiliary mechanisms rather than additional principal layers of the design space.

4 From Problem Characteristics to PINN Design

This section develops the problem-characteristic vocabulary used to key recommendations in Sections 5 and 7, and maps these characteristics onto the design choices constructed above. A characteristic is included only where at least one source demonstrates that it changes which design choice is preferable. Figure 3 summarizes this mapping across the four characteristic categories developed in **??** and sections 4.1 and 4.2; Table 3 provides the corresponding characteristic-by-characteristic detail.

PDE/ODE order. Higher-order derivatives entering the residual can motivate a gradient-enhanced loss, which augments (3) with a penalty on the residual's own

Table 2 PINN design-space taxonomy organized by layer, with representative sub-categories and supporting references.

Layer / category	Sub-category	Reference	Notes
Formulation (physics layer)	Strong	[1]	Default; sensitive to solution roughness
	Weak/Galerkin	[21]	Requires quadrature; lower differentiability requirement; *not* the same concept as weak enforcement below
	Energy	[22]	Requires a variational principle; distinct from the weak form
Enforcement (physics layer, coupled to formulation)	Soft	[1]	Default; weighting hyperparameter sensitive
	Hard	[27]	Best-supported default where exact satisfaction is required [29]; direction corroborated by [31–34], with counterexamples in [35] and one sub-case of [29] (Section 3.2)
	Weak (Nitsche)	[29]	Requires a weak-form residual; unavailable under strong form; distinct from weak formulation above
Architecture (representation layer; not inherently physics-informed)	MLP	[1]	Default backbone
	CNN	[39]	Grid/geometry-mapped domains
	RNN/LSTM	—	Sequential/temporal structure
	GNN	—	Irregular/relational domains
	Attention/ Transformer	[68]	Temporal-dependency capture (Section 8.2)
	KAN/PIKAN	[69]	Matched-protocol comparison against MLP; no uniform winner. Held at E3: the three MLP-versus-KAN comparisons share a research group (Section 8.1)
Operator/derivative evaluation (computational-realization layer)	Reverse-mode AD	[41]	Default; exact to floating-point precision
	Forward-mode/ separable/ Taylor-mode AD	[42, 43]	Derivative stays exact; evaluation order and memory change. Reduced cost in high-dimensional and high-order settings
	Finite diff./ finite volume/ local fitting	[44, 47]	Comparable or better accuracy at lower cost across several non-affiliated studies (Section 5.5); correctness fix in [7, 48]
	Differential quadrature/ RBF-FD	[48]	Positioned between AD and fixed stencils: cheap high-order derivatives on scattered points
	Quadrature (integration)	[21]	Required by, not independent of, weak/energy formulation

spatial derivatives,

$$\mathcal{L}_r^g(\theta) = \mathcal{L}_r(\theta) + \mu \frac{1}{N_r} \sum_{k=1}^{N_r} \left| \nabla_x \mathcal{N}[u_\theta; \lambda](x_k, t_k) \right|^2, \tag{11}$$

as defined in Equation (11), originally proposed as gPINN [70] and shown to reduce error on a fourth-order benchmark problem [17]. Thus, PDE order affects both formulation and operator-evaluation cost.

Nonlinearity and stiffness. Nonlinearity with sharp fronts has been associated with a preference for a standard rather than gradient-enhanced formulation, although the result is not uniform across the nonlinear problems tested [17]. For stiff, time-dependent problems, exact initial-condition enforcement is reported to have a stronger effect on training stability than the tested self-adaptive loss weighting [34]. A hybrid quantum-classical study reports its largest gains over classical baselines in stiff and multiscale regimes [71], but this finding remains uncorroborated (Section 8.7). The evidence therefore points more clearly to stiffness than to nonlinearity alone as a driver of these design choices.

High-frequency and oscillatory behavior. High-frequency content is associated with spectral-bias effects [72, 73] and can alter the relative performance of enforcement mechanisms [29]. Reported responses include multilevel domain decomposition [57], hard enforcement combined with a multi-scale subnetwork and Fourier-induced activations [33], and exact initial-condition enforcement [34]. Thus, high-frequency behaviour can affect enforcement, architecture, and domain decomposition rather than mapping to a single design choice.

Sharp gradients and discontinuities. Sharp features are central to shock-capturing comparisons [18] and are also associated with cases where lower training loss does not imply greater solution accuracy [74]. Reported responses include gradient-annihilating loss weighting for relativistic Riemann problems [75], cusp-enforcing inputs for elliptic interface problems [76], and residual-and-gradient-based adaptive sampling for sharp solutions [60]. Sharp features therefore motivate changes in loss, representation, and sampling rather than a single architectural choice.

Multiscale behavior. Multiscale behaviour motivates spatial domain decomposition [53, 54, 57] and temporal decomposition [77]. A finite-volume residual is also reported to recover multiscale features of higher-Reynolds-number cavity flow that an automatic-differentiation PINN did not [47]. Thus, multiscale behaviour can affect both decomposition and computational realization.

Conservation laws and variational structure. The existence of a suitable variational structure makes the energy formulation of Section 3.1 applicable, and this formulation is favored "when applicable" in structural-engineering applications [78, 79]. The design implication is therefore conditional on the governing problem admitting such a representation.

Constitutive singularities. A granular-flow reconstruction study addresses a viscosity singularity at vanishing shear rate through regularization of the governing law and a two-phase training schedule [80]. This is retained separately because the documented response modifies the governing physics rather than selecting among the PINN design layers.

4.1 Computational characteristics

Dimensionality. High dimensionality motivates separable and forward-mode differentiation and stochastic dimension-wise gradient methods [14]. Trace estimation

provides another response by replacing explicit Hessian construction with Hessian–vector products [43], while adaptive sampling can relocate rather than add collocation points to reduce the cost of high-dimensional resampling [61]. Dimensionality therefore primarily affects operator evaluation and sampling.

Domain complexity and geometry. Complex or irregular geometries motivate domain decomposition [53, 54] and geometry-specific hard-enforcement constructions [27]. They also constrain numerical operator evaluation: meshless radial-basis-function finite differences on point clouds [44], signed-distance-function constructions that prevent stencils from crossing boundaries [46], and differential quadrature on scattered points [48] provide alternatives for irregular domains. Geometry can therefore affect decomposition, enforcement, and operator evaluation.

Temporal scale. Temporal scale motivates temporal domain decomposition [77] and causal-weighting schemes [81]. For stiff time-dependent problems, exact initial-condition enforcement has additionally been reported to improve training stability [34]. Temporal scale therefore affects both domain decomposition and the allocation of training emphasis across time.

Computational budget. The retained literature does not establish a transferable cost-versus-accuracy mapping across design choices. Individual studies nevertheless report two- to four-fold faster training at comparable loss [44], roughly one tenth of the training time at higher accuracy [47], and strong and weak scaling for a decomposition hierarchy [57]. These results demonstrate computational differences, but cannot yet be compared on a common cross-design scale. This limitation is revisited in Section 9.

4.2 Data characteristics

Sparse or noisy observational data. Sparse and noisy data are central to inverse and data-assimilation settings [4, 9]. A ship-maneuvering identification study directly reports that a physics-informed model retained high accuracy under noise levels at which a purely data-driven baseline diverged [82]. More generally, learning from scarce data is not unique to PINNs [83], but physics-informed regularization provides one documented response. The main implication is that physical constraints can provide useful regularization when observations are sparse or noisy.

Forward, inverse, and data-assimilation settings. Following [1, 14], a problem is forward when parameters and boundary/initial data are specified; inverse when some quantities must be inferred jointly with the solution; and data assimilation when a partially known model is reconciled with observations. These settings are treated as data characteristics rather than separate design layers because they change the loss terms and trainable quantities while the underlying formulation, enforcement, architecture, and computational-realization choices can remain the same. Theoretical analysis supports this treatment for a class of inverse PDE problems [84] (T). However, an independent benchmark finds that weak form outperforms strong form on inverse problems in its test suite, with no comparable advantage on forward problems [85]. Thus, problem setting can interact with formulation choice without constituting a separate design layer.

4.3 Occupancy of the design space

Two entries in Table 2 illustrate why occupancy must be distinguished from simple absence of evidence. First, weak/Nitsche enforcement under a strong-form residual is structurally inaccessible: Nitsche's method operates on the weak-form residual and therefore has no corresponding pointwise strong-form interpretation [29]. Second, the hard×energy combination is supported by two non-affiliated fracture studies, both of which report difficulties with soft enforcement and use exact boundary satisfaction [7, 23]. This provides corroborated occupancy for that combination (Section 5.3).

Distinguishing structurally inaccessible combinations from those for which evidence is simply absent matters practically: the former are excluded by the mathematical structure of the method, whereas the latter remain potential targets for future comparative work. A full three-way occupancy audit crossing formulation, enforcement, and operator evaluation has not been attempted here and is noted as a limitation in Section 9. Figure 4 therefore focuses on the formulation×enforcement projection, for which the most direct comparative evidence currently exists, principally [29].

4.4 Overall mapping

Taken together, the mapping shows that problem characteristics rarely select a single universally preferred PINN design. Mathematical characteristics primarily affect formulation and enforcement, while computational characteristics often affect operator evaluation, sampling, and decomposition. Data characteristics mainly change the composition and role of the training objective. Several characteristics, particularly high-frequency behaviour, sharp gradients, multiscale structure, and complex geometry, span multiple layers. The design space should therefore be read as a conditional mapping from problem characteristics to candidate design choices, rather than as a fixed hierarchy with one prescribed PINN design for each problem.

5 Comparative Evidence and Trade-offs

5.1 Grading the evidence

Throughout this survey, every comparative claim is assigned one of five grades, defined once here and used without further explanation in subsequent tables. **E1** denotes the strongest empirical evidence used here: a matched-protocol, multi-method benchmark, or independent controlled studies whose protocols are sufficiently matched to support the same directional conclusion. **E2** denotes directional corroboration by a non-affiliated study or group when the studies do not share a fully matched protocol. **E3** denotes a finding supported primarily by one study or one research group. **T** denotes a theoretical or analytical result without broad empirical validation. $\emptyset$ denotes insufficient evidence for a general recommendation. These grades describe the *strength and independence of the evidence for a claim*, not methodological quality, and are not increased by venue, citation count, recency, or additional papers from the same research group. Table 4 summarizes the grades with representative examples from the sections that follow.

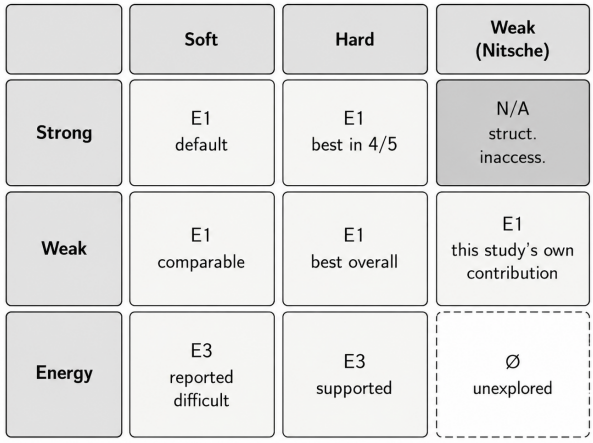


Fig. 4 Evidence/occupancy matrix for formulation (rows) and enforcement (columns). E1/E2 denote established or corroborated evidence, E3 single-study evidence; the dark cell is structurally inaccessible, while the dashed cell represents an unexplored combination.

Table 3 Problem characteristics relevant to PINN design and the design considerations they affect.

Characteristic	**Relevance to design**
PDE/ODE order	Favors a gradient-enhanced residual term for higher-order operators
Nonlinearity, sharp fronts	Favors a standard over a gradient-enhanced formulation, non-uniformly
Stiffness, multiscale dynamics	Favors exact initial-condition enforcement for training stability; reported as favorable for hybrid quantum-classical approaches in one study
High-frequency/oscillatory content	Changes which enforcement mechanism retains a theoretical convergence rate; addressed independently at the enforcement, architecture, and decomposition layers
Sharp gradients/discontinuities	Organizes an entire technique taxonomy; loss and accuracy can decouple; favors loss-modification, input-feature, and gradient-aware sampling responses over backbone changes
Multiscale spatial behavior	Motivates spatial, multilevel, and temporal domain decomposition
Variational/energy structure available	Precondition for, and favors, the energy formulation
Constitutive singularity	Requires physics-level regularization rather than a formulation/architecture choice
Dimensionality	Motivates forward-mode/separable/Taylor-mode differentiation, trace estimation, and dimension-wise or point-relocating sampling
Domain/geometry complexity	Motivates decomposition and geometry-specific hard constructions; also forces meshless rather than stencil-based numerical differentiation
Temporal scale	Motivates temporal decomposition and causal weighting
Computational budget	Individual cost figures and one scaling study exist, but no accuracy-versus-cost mapping on a shared problem and protocol
Sparse/noisy data	Favors physics-informed models over purely data-driven baselines
Problem setting (forward/inverse/assimilation)	Primarily determines loss composition and trainable quantities; one benchmark also reports a formulation-level advantage for the weak form specifically on inverse problems

Table 4 Evidence grades used throughout this survey. The grade records how strongly and how independently a claim is corroborated, not how well the underlying study was conducted; methodological quality is assessed separately against Table 5.

Grade	Criterion	Example
E1	Matched-protocol multi-method benchmark, or corroboration across two or more non-affiliated controlled studies with sufficiently matched protocols	The four-method enforcement comparison of [29]
E2	Directional corroboration by a non-affiliated study or group without a fully shared protocol	Loss/accuracy decoupling, reported independently in [29] and [74]
E3	Finding supported primarily by one study or one research group	Most individual variant-introduction results in Sections 5.3 to 5.5
T	Theoretical or analytical result without broad empirical validation	The error-decay estimates of [30]
∅	Insufficient evidence for a specific recommendation	Several rows of Tables 3 and 6

Methodological quality is assessed separately using the comparability checklist in Table 5.

5.2 Strength and Comparability of the Evidence

PINN variants have been compared extensively, but the published comparisons are not uniformly matched in experimental design or reporting. Across the retained comparative studies, architecture, optimization, training budget, random seeds, and reporting of variance are not consistently controlled or reported. The resulting comparisons therefore provide useful directional evidence, but cannot always be placed on a common quantitative scale.

Several studies demonstrate stronger comparative practice. A 22-case PDE benchmark spanning ten PINN variants uses a matched architecture and training budget and reports three independent runs with mean and standard deviation [85]. An enforcement comparison fixes architecture and optimizer across four methods and reports variance information across five problem classes [29]. A dam-break flow study evaluating four training enhancements and their combinations reports ten independent runs with variance [74]. These studies show that matched comparisons are feasible, but reporting practices remain uneven across the literature.

Other studies illustrate the resulting limitations. The shock-front benchmark varies architecture and training configuration across the techniques compared and reports neither seed counts nor variance estimates [18]. The PINN-versus-gPINN benchmark fixes its architecture across eight problems but uses a single training run per configuration; in its Helmholtz interface case, the reported result favours the standard PINN while the decision table assigns the case to gPINN [17]. This illustrates how a

Table 5 Minimum reporting and comparability checklist for comparative PINN studies.

Item	Why it matters
Architecture and parameter count per method	Identifies architectural and parameter-budget differences that may confound accuracy or efficiency comparisons
Optimizer and schedule	Identifies optimization differences that may affect convergence and final performance
Collocation/data point counts	Identifies differences in the amount of information supplied during training
Error metric, precisely defined	Distinguishes training loss from solution-error measures (Section 6)
Training budget	Separates convergence speed from final or asymptotic accuracy
Hardware and wall-clock cost	Enables comparison of accuracy relative to computational cost
Independent runs and variance	Distinguishes repeatable effects from single-run results and quantifies variability
Hyperparameter-tuning parity	Prevents differences in tuning effort from confounding the comparison

summary recommendation can obscure conflicting evidence within the same comparison. The multi-task PINN and Bayesian uncertainty-quantification studies likewise use single runs per configuration despite reporting controlled ablations [86, 87].

Comparability also depends on independence. The three studies comparing Kolmogorov–Arnold and multilayer-perceptron representations at matched parameter count share authors [66, 69, 88]; they therefore provide multiple comparisons but not independent research-group replication. By contrast, the derivative-evaluation study of [49] reports six independent seeds. The evidence grades consequently consider both experimental comparability and independence rather than simply counting published comparisons.

This unevenness is consistent with broader concerns about weak baselines and reporting bias in machine learning for fluid-related PDEs [89]. The evidence synthesized here is therefore interpreted primarily through directional agreement and experimental comparability rather than pooled numerical rankings.

Table 5 proposes a minimum reporting checklist for comparative PINN studies, informed by matched-protocol, seed/variance, and hardware/cost reporting in representative studies [66, 69, 85]. The checklist is a recommendation for future comparative work, not a description of current practice.

5.3 Formulation and enforcement (physics layer)

Formulation and enforcement are closely coupled physics decisions and are therefore synthesized jointly. The comparative evidence does not identify a universally preferred enforcement mechanism. In the controlled comparison of [29], the four enforcement methods performed comparably on the more regular elliptic case, while soft enforcement was marginally more accurate on one convection-dominated case. Other studies

report advantages for projection-based hard constraints [31], augmented-Lagrangian enforcement [32], hard output transformations [33], and exact initial-condition transformations for stiff problems [34]. Conversely, fully soft initial and boundary conditions produced the lowest error among four combinations in an acoustic-wave study [35]. The evidence therefore supports hard enforcement as a strong conditional default when exact boundary or initial satisfaction is required and an admissible construction exists, but not as a universal rule.

Formulation choice shows a similarly conditional pattern. Where a variational or energy principle is available, two independent fracture studies support energy formulations with exact boundary enforcement, reporting improved accuracy and reduced derivative requirements relative to residual-based formulations [7, 23]. However, pure energy and strong formulations can miss stress-concentration features, motivating mixed energy formulations [24]. A benchmark across eight PDE classes reports advantages for gradient-enhanced formulations in some high-order settings but nonuniform results across nonlinear cases [17], while a 22-case benchmark finds a weak/variational formulation substantially better for inverse problems but not for the corresponding forward problems [85]. Thus, formulation choice should be conditioned on the mathematical and task characteristics of the problem.

Exact enforcement can remove boundary-condition penalty tuning and improve accuracy or stability in some settings, but requires an admissible construction and is not uniformly superior. Energy and weak formulations can reduce derivative requirements or improve inverse-problem performance when their structural assumptions are satisfied, but may introduce additional quadrature requirements or remain sensitive to localized solution features.

Formulation and enforcement should be selected jointly. Hard enforcement is a strong conditional default when exact boundary or initial satisfaction is required and an admissible construction exists; energy or weak formulations become attractive when the problem structure supports them. No universal formulation–enforcement combination is supported by the available evidence.

5.4 Architecture (representation layer)

Architecture is a representation choice whose comparative performance depends on the problem and training configuration. A matched-parameter comparison of multilayer perceptron and Kolmogorov–Arnold representations finds no uniform winner, with performance depending on the problem and configuration and with spline-based representations carrying higher per-parameter computational cost in some settings [69]. Two further matched-parameter comparisons report similarly configuration-dependent outcomes, with the relative performance of KAN and MLP representations varying with optimizer, numerical precision, and parameter budget [66, 88]. These studies share authors and therefore do not provide independent research-group corroboration; the architecture-level finding remains E3.

For discontinuous or shock-dominated problems, the comparative evidence does not support a general architectural solution. A shock-capturing benchmark reports that architecture modifications were unsuccessful on the tested shock structures,

whereas physics- and loss-modification approaches were more effective [18]. An independent study reports that gradient-annihilating loss weighting achieves accuracy comparable to a second-order shock-capturing scheme and substantially better than a baseline PINN on relativistic Riemann problems [75]. This finding is problem-specific: a cusp-enforcing level-set input successfully resolves an elliptic interface feature in another study [76].

Alternative architectures may provide useful representational capacity for particular structures, but parameter count, optimizer, numerical precision, and training configuration can materially change the comparison. Architectural modification is therefore not necessarily the most effective response to non-smooth solutions; targeted changes to the loss, physics representation, or input construction may be preferable.

No architecture is currently supported as a universal PINN backbone. Representation choice remains problem- and configuration-dependent, with the strongest comparative evidence favouring conditional rather than general architectural recommendations.

5.5 Computational realization (operator/derivative evaluation)

This layer concerns how an already-defined differential operator is computed. Multiple studies compare automatic differentiation (AD) with numerical or restructured alternatives, including meshless finite differences, finite-volume residuals, differential quadrature, trace estimation, and hybrid derivative schemes [43–49]. Across these studies, appropriately designed numerical alternatives are reported to achieve comparable or improved solution accuracy at lower training cost in the tested problems, with reported examples ranging from two-to-four-fold faster training to roughly one-tenth of the training time [44, 45, 47]. This provides E2 evidence that AD need not be the computational default when a suitable numerical realization is available.

The comparison also extends beyond computational cost. In an energy-formulation fracture problem, finite-element-style numerical gradients correct qualitatively incorrect AD-based solutions while also reducing training cost [7]. Independently, differential quadrature resolves higher-Reynolds-number flow cases on which the AD-PINN struggles and is also reported to avoid nonphysical solutions under sparse sampling [48]. These are problem-specific cases rather than evidence that AD is generally unreliable.

The alternatives also involve different trade-offs. Meshless and hybrid schemes can address irregular geometries, while separable evaluation and trace estimation reduce the cost or memory requirements of higher-order differentiation [42–44, 46, 48]. Numerical schemes, however, introduce dependence on discretization, point distribution, boundary treatment, and operator order. The current evidence therefore does not establish a ranking among finite-difference, finite-volume, differential-quadrature, and other numerical realizations.

A remaining gap is the absence of a shared-protocol benchmark that varies only the derivative-evaluation mechanism while holding formulation, architecture, sampling, and training budget fixed. The present evidence therefore supports a directional preference for appropriately designed numerical alternatives in some settings, but not

a universal replacement for automatic differentiation. This gap is carried forward to Section 9.

Therefore, derivative evaluation is a substantive PINN design choice rather than a purely implementation detail. Numerical realizations can match or improve accuracy at lower training cost and have corrected qualitatively incorrect AD-based solutions in particular settings (E2), but their suitability depends on the operator, geometry, sampling, and computational setting. No universal ranking of AD and numerical alternatives is currently supported.

5.6 Central comparative synthesis

Table 6 consolidates the principal conditional recommendations from the comparative evidence into a decision-oriented evidence map. Each row links a problem characteristic to a candidate design choice, the principal alternative, supporting evidence, and an associated trade-off. Evidence grades apply to the specific proposition stated in each row and should not be interpreted as universal rankings of PINN variants.

Table 6: Central synthesis of comparative evidence for PINN design choices. Evidence grades describe the strength and independence of support for each proposition.

Problem characteristic	Design choice	Alternative	Key evidence	Grade	Trade-off / limitation
Physics layer: formulation and enforcement					
Regular elliptic problem	No preferred enforcement	Soft / hard / weak	Four methods comparable in the regular elliptic case [29]	E1	No decisive advantage
Exact boundary conditions required; admissible construction exists	Hard enforcement	Soft penalty / weak–Nitsche	Best in 4/5 classes in a matched comparison; supported elsewhere by [31–33]	E1	Geometry/problem dependent
Convection-dominated or point-source wave problems	Soft enforcement	Hard enforcement	Soft marginally better in one variational case; fully soft best of four combinations in an acoustic-wave study [29, 35]	E2	Hard enforcement not universally optimal
Variational / energy principle; exact BC required	Energy formulation + exact enforcement	Strong residual formulation	Deep Ritz fracture studies use exact BCs; energy formulations reduce derivative order and improve accuracy in direct comparisons [7, 23]	E2	Concentration features may require mixed formulations [24]
High-order differential operator	Gradient-enhanced residual	Standard residual	Lower error on the tested fourth-order benchmark [17]	E3	Extra derivative/training cost; single study
Inverse problem	Weak / variational formulation	Strong formulation	Stronger performance for inverse, but not corresponding forward, cases in a 22-case benchmark [85]	E3	Single benchmark; problem-setting dependent
Representation layer: architecture					
Architecture choice, general	No universal preference	KAN/PIKAN vs. MLP	Matched comparisons show configuration-dependent outcomes [66, 69, 88]	E3	Depends on representation, optimizer, and parameter budget
Shock-dominated problem	Loss modification	Architecture modification	Architecture modifications ineffective in tested shock benchmarks; loss modification achieved comparable accuracy to a shock-capturing scheme [18, 75]	E2	Not general to all non-smooth solutions; cusp features can be resolved architecturally [76]
Computational-realization layer: operator evaluation					
Derivative evaluation is a major training-cost bottleneck	Numerical derivative / operator scheme	Automatic differentiation	Several studies report comparable or improved accuracy at lower cost [44–48]	E2	Depends on operator, geometry, and discretization; no universal ranking
Problem-specific cases of AD failure	Numerical derivative evaluation	Automatic differentiation	Qualitatively incorrect AD solutions reported in fracture and high-Reynolds-number flow settings [7, 48]	E2	Failure frequency unknown
Auxiliary mechanisms					

continued on the next page

Table 6, continued from the previous page.

Problem characteristic	Design choice	Alternative	Key evidence	Grade	Trade-off / limitation
Large, complex, or multiscale domain	Domain decomposition	Single monolithic network	Improves scalability or solution quality in tested complex and multiscale settings [53, 54, 56]	E2	Interface conditions and partitioning add complexity
High-frequency / multiscale spatial solution	Multilevel decomposition	Single-level decomposition / single network	Improved performance and scaling on tested high-frequency and multiscale solutions [57]	E3	Limited independent evidence
Sharp solution features	Adaptive collocation sampling	Fixed / uniform sampling	Improves reported accuracy or efficiency in tested sharp-solution problems [59–61]	E2	Resampling costly in high dimension
Competing loss terms of unequal scale	Adaptive loss balancing	Fixed manual weights	Relative balancing gave lowest reported error among four schemes in one comparison [64]	E3	Ranking depends on loss and activation design [65]
Second-order optimization is computationally affordable	Curvature-aware / quasi-Newton optimization	Adam or standard first-order / quasi-Newton methods	Two independent studies report improved performance with curvature-aware optimization [66, 67]	E2	Cost remains problem- and implementation-dependent; no ranking among second-order methods
Data characteristics					
Noisy inputs and outputs	Joint input/output noise modelling	Output-noise-only model	Lower error and improved parameter inference across six problems [87]	E3	Benefit depends on relative input/output noise

6 Interactions Between Design Choices

The design choices synthesized above are not independent. Several studies show that the effect of one choice can depend on another, so the evidence cannot always be reduced to a one-property-to-one-recommendation mapping. These interactions motivate the ordered decision procedure in Section 7.

Formulation interacts with enforcement *(established).* The ranking among enforcement mechanisms depends on formulation and problem setting: soft enforcement marginally outperforms hard enforcement under a weak formulation on one hyperbolic problem, while hard enforcement is best in four of five problem classes tested overall [29]. This supports treating formulation and enforcement as coupled decisions.

Formulation interacts with problem setting *(plausible, single-study).* A 22-case benchmark reports a weak/variational formulation outperforming the strong form on inverse problems, with no comparable advantage on the corresponding forward problems [85] (Section 5.3). Thus, formulation choice can depend on whether the task is forward or inverse.

Architecture interacts with optimizer choice *(plausible, single-study).* A physics-informed Kolmogorov–Arnold representation that underperforms an MLP at matched parameter count closes most of the gap when given more parameters and a self-scaled quasi-Newton optimizer rather than Adam [66]. Because the choices are varied jointly, their individual contributions cannot be isolated from this study alone.

Loss weighting interacts with activation function *(plausible, single-study).* A comparison of loss-balancing schemes with trainable activation functions reports that the ranking of weighting strategies depends on loss design and the network's function-approximation capability [65]. Weighting schemes should therefore not be interpreted as architecture-independent choices.

Enforcement interacts with stiffness *(plausible, single-study).* An ablation on stiff, time-dependent PDEs reports that exact initial- condition enforcement substantially improves training stability and has a larger effect than the tested self-adaptive loss weighting [34]. The evidence remains limited to the tested problem class.

Training loss and solution accuracy are not equivalent *(E2 corroborated).* A shallow-water dam-break study shows that training loss and solution accuracy do not necessarily track together [74]. An independent enforcement comparison reports the same qualitative behaviour [29]. Training loss alone should therefore not determine model selection; solution accuracy should also be assessed.

Two dependencies are structural rather than empirical: weak and energy formulations require quadrature, while numerical derivative schemes for irregular domains may require meshless constructions. These constraints affect admissibility rather than establishing a performance ranking (Section 5.5).

Accordingly, the decision framework applies design choices sequentially, conditioning later choices on problem characteristics and earlier design decisions rather than treating the design space as a flat lookup table.

7 A Problem-Driven PINN Design Procedure

7.1 Procedure

Figure 5 presents the decision procedure developed in this survey. It moves from problem characterization to structurally admissible design choices and then to comparative evidence. Where no specific comparative ranking exists, conventional PINN principles and structural admissibility guide the selection.

Q1. Is the problem forward, inverse, or data-assimilation, and what data are available (Section 4.2)?

Q2. What is the governing equation's mathematical character – order, linearity, stiffness, sharp fronts or discontinuities, and availability of a variational or energy principle (**??**)?

Q3. What boundary and initial conditions must be satisfied, and is exact enforcement required (Section 3.2)?

Q4. What is the domain's geometric and multiscale complexity, including whether it is simple, complex, or naturally decomposable (Section 4.1)?

Q5. What computational constraints apply, including dimensionality, derivative cost, hardware, and training budget (Section 4.1)?

Q6. Which formulation and enforcement mechanisms are structurally admissible for the identified problem characteristics (Section 4.3 and table 2)?

Q7. Given the admissible choices, what does the comparative synthesis and interaction evidence support, and at what evidence grade (Table 6 and section 6)?

Q8. Does the evidence support a general recommendation for the identified combination of characteristics? If not, should an emerging direction in Section 8 be explored, or should the best-supported baseline be adopted while reporting the evidence gap?

The selected configuration is then validated on the target problem and reported using Table 5. Where no general recommendation is supported, the evidence gap is reported explicitly.

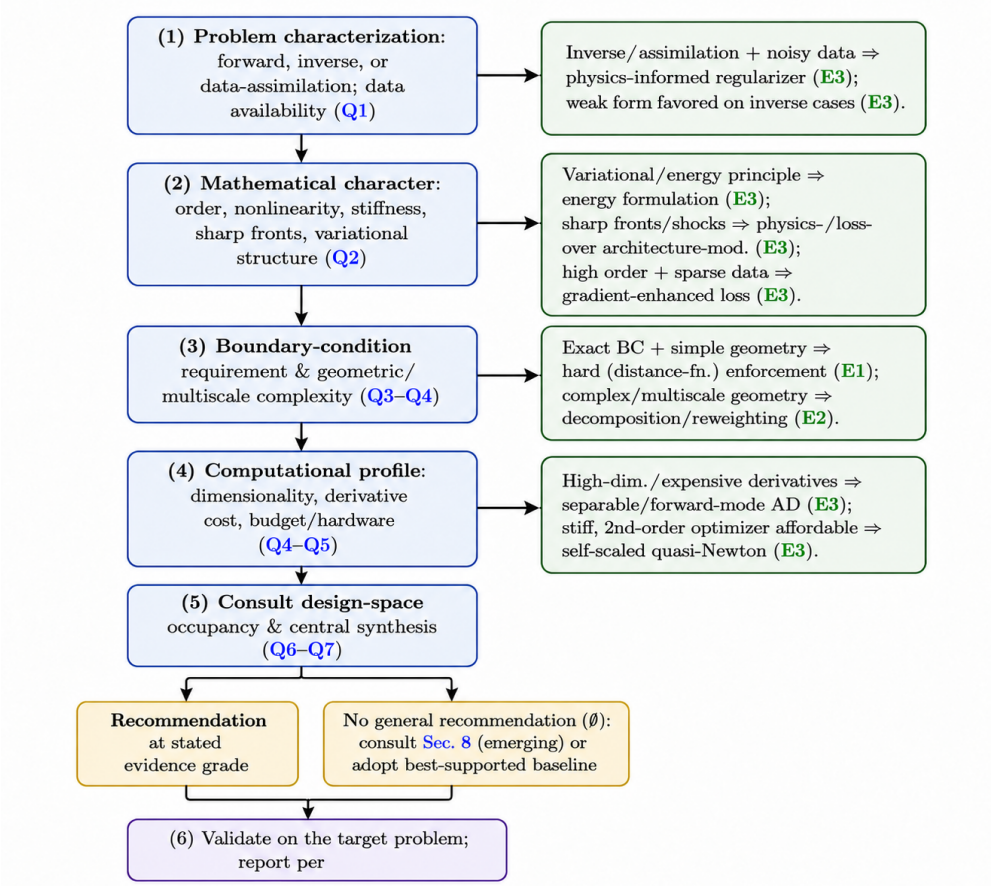


Fig. 5 Problem-driven PINN design procedure. The workflow moves from problem characterization through mathematical, boundary/geometric, and computational considerations to an evidence-graded design recommendation, with validation on the target problem as the final step.

7.2 Worked examples

Example 1: forward heat conduction on a complex geometry. A forward problem (Q1) with a smooth solution, second-order operator, and exact Dirichlet data on a geometrically complex domain (Q2–Q4). Since the geometry admits a distance-function construction, Q6 identifies the strong formulation with hard enforcement as structurally admissible and supported at grade E1 by Table 6. With no unusual computational constraint (Q5), a conventional MLP with reverse-mode automatic differentiation provides a suitable baseline. The result is then validated using Table 5.

Example 2: inverse ship-maneuvering identification from noisy data. An inverse problem (Q1) with sparse, noisy trajectory data and a known but partially uncertain governing model (Q2). Q7 identifies evidence that a physics-informed model retains accuracy under tested noise levels where a purely data-driven baseline degrades [82] (E3). The procedure therefore favours a standard physics-informed

identification model, with the physical model providing a constraint on the noisy observations.

Example 3: fracture propagation with an energy principle and exact displacement boundaries. A forward problem (Q1) with an available variational energy principle and a requirement that displacement boundary conditions hold exactly (Q2–Q3). Q6 identifies the energy formulation with hard enforcement as structurally admissible and supported at grade E2, with independent fracture studies reporting advantages over residual-based formulations [7, 23]. Q7 also identifies numerical derivative evaluation as a relevant computational choice: finite-element-style numerical gradients corrected a documented AD failure in one fracture study [7], consistent with an independent AD failure case in high-Reynolds-number flow [48] (Section 5.5). If stress-concentration features are present, a mixed energy formulation provides a documented alternative [24].

8 Emerging Directions

The three layers of Section 3—formulation and enforcement, architecture, and computational realization—define the core PINN design space considered in this survey. This section examines seven emerging or adjacent directions that extend this space: KAN/PIKAN, Transformers, operator learning, adaptive and automated design, data-assisted and uncertainty-aware PINNs, scalable PINNs, and quantum and hybrid quantum-classical PINNs. They represent newer architectures, learning paradigms, automated design strategies, data and scaling extensions, or alternative computational substrates. Their evidence maturity is summarized in **??**.

8.1 Kolmogorov–Arnold and physics-informed KAN architectures

Kolmogorov–Arnold networks (KANs) replace fixed activation functions with learnable univariate functions associated with network edges. When combined with the physics-informed formulation and enforcement machinery, they form PIKANs and therefore modify the representation layer rather than introducing a new physics formulation.

Matched-parameter comparisons find no uniform advantage over MLPs and report higher computational cost per parameter for spline-based KANs in some configurations [69] (E3). Further studies report configuration-dependent outcomes: PIKANs underperform MLPs at equal parameter count but reduce the gap with more parameters and a different optimizer [66], while representation and optimizer choices interact in systems-pharmacology models [88]. These comparisons share authors and therefore do not provide independent research-group corroboration. Recent work includes a neural-tangent-kernel analysis of Chebyshev-based PIKANs [90] (T) and a residual-gated deep PIKAN that outperforms parameter-matched Chebyshev PIKANs and PirateNets on nine forward benchmarks [91]. The latter compares variants within the KAN family rather than establishing an advantage over MLP-based PINNs.

The available evidence therefore does not establish a general accuracy or efficiency advantage for KAN/PIKAN. Performance depends on parameter budget, optimizer,

precision, and representation details, while deeper PIKANs have also been reported to become unstable [91]. KAN/PIKAN is best treated as an alternative representation whose suitability remains problem-dependent.

8.2 Transformer and attention-based physics-informed models

Transformer and attention-based architectures augment the pointwise representation of standard PINNs with mechanisms that model interactions between spatial or temporal points. Examples include physics-informed transformer networks [92], transformer-based PINN frameworks [68], and transformer PDE solvers designed for general geometries [93]. Attention does not itself make a model physics-informed; the governing equations and constraints remain part of the formulation and enforcement mechanisms.

The main motivation is to capture long-range spatial or temporal dependencies that may be difficult to represent efficiently with a pointwise MLP. Existing studies demonstrate this feasibility and report applications involving temporal dependencies and complex geometries [68, 92, 93] (E3), but a controlled comparison with standard MLP-based PINNs under matched parameters, training budgets, and problem settings has not been identified for this survey. Attention has also been used across task branches in a multi-task PINN [86], although this addresses task-level rather than collocation-point interactions.

Full self-attention can become computationally expensive as the number of points increases, and the current evidence does not establish an accuracy, efficiency, or scaling advantage over MLP-based PINNs. Attention-based PINNs are therefore a plausible emerging option for problems with pronounced long-range dependencies rather than a general replacement for MLPs.

8.3 Operator learning: DeepONet, Fourier neural operators, and PINO

DeepONet and Fourier neural operators (FNOs) learn mappings between function spaces rather than a single solution for one problem instance. DeepONet uses a branch–trunk architecture [36], while FNO uses spectral convolutions [37]. Their original data-driven forms do not require a physics-informed loss; physics-informed neural operators (PINOs) incorporate physics-based constraints such as PDE residuals or energy terms [38].

The principal motivation is amortization: after training, an operator can be evaluated on new instances from the same problem family without retraining a separate network. This is therefore most relevant to repeated solutions of related PDEs, such as parametric studies, design exploration, or uncertainty quantification.

Physics-informed operator learning has been demonstrated through physics-constrained DeepONet [38] (E3), establishing feasibility but not a general advantage over conventional PINNs. Direct matched comparisons between physics-informed operator learners and standard PINNs remain limited. Operator learning also requires representative training instances, and its additional training cost is justified only when the learned operator is reused sufficiently often. PINO should therefore be viewed as

an extension of physics-informed learning to repeated problem families rather than a general replacement for instance-specific PINNs.

8.4 Adaptive and automated PINN design

Adaptive and automated PINN design applies automated search or adaptation to choices otherwise selected manually. NAS-PINN uses neural architecture search [94], while Auto-PINN applies automated hyperparameter optimization [95]; evolutionary approaches to related design problems have also been surveyed [96]. Adaptive loss weighting is a related but distinct mechanism. A multi-task PINN combines cross-flow attention with dynamic allocation of loss weights across three Navier–Stokes regimes [86].

The motivation is to reduce manual trial and error in a design space with many interacting choices. Automated architecture and hyperparameter search have been demonstrated in individual studies [94, 95] (E3), but controlled comparisons against well-tuned expert-designed PINNs remain limited.

Adaptive loss weighting has a broader comparative basis. A comparison of learning-rate annealing, GradNorm, SoftAdapt, and relative-loss balancing across forward and inverse formulations of three benchmark PDEs reports relative-loss balancing as the most accurate of the tested methods, with lower computational overhead in those experiments [64]. Related self-adaptive weighting approaches have also been evaluated [63, 97]. However, a later study finds that the relative ranking depends on loss design and activation function [65]. Adaptive weighting is therefore supported as a useful option, but not as a universally preferred scheme.

Automated search introduces additional computational cost and can explore only a selected portion of the design space. The multi-task result is also limited to three closely related 2D flow problems [86]. These approaches are most relevant when the potential improvement in configuration quality justifies the additional search or adaptation cost.

8.5 Data-assisted and uncertainty-aware PINNs

Data-assisted PINNs incorporate observational data into physics-informed models, while uncertainty-aware formulations additionally represent uncertainty arising from noisy or incomplete observations. Bayesian PINNs provide a probabilistic formulation for inference under noisy data [98], while related approaches model uncertainty in both observed outputs and input coordinates [87].

These methods are particularly relevant to inverse problems and data-assimilation settings where observations are sparse or noisy and physical constraints provide additional information. Bayesian PINNs demonstrate uncertainty-aware inference from noisy observations [98] (E3). An extension modelling noise in both input coordinates and observed outputs reports improved parameter inference over an output-noise-only B-PINN in tested inverse problems and demonstrates applicability to pretrained neural operators [87] (E3).

The evidence supports explicit uncertainty modelling when measurement uncertainty is important, but does not establish a generally superior uncertainty-quantification method. Bayesian inference can introduce substantial computational overhead, and the benefit of modelling input uncertainty depends on the relative magnitude and structure of input and output noise [87]. Direct comparisons with alternatives such as ensembles or dropout on common benchmarks remain limited.

8.6 Scalable and distributed PINNs

Scalable PINNs use domain decomposition to represent and train solutions through multiple coupled sub-networks. Conservative and extended PINNs (cPINN and XPINN) [52, 53], finite-basis PINNs (FBPINN) [54], and parallelized variants [55] partition problems across subdomains and can distribute the associated computation.

The main motivation is to address large, geometrically complex, or multiscale problems for which a single network becomes difficult to train. Domain decomposition has demonstrated benefits across several PINN formulations [52–54, 56]. A 22-case comparative benchmark independently reports benefits of decomposition-based approaches for complex-geometry and multiscale problems [85] (E2). Multilevel decomposition further reports improved accuracy relative to a single network and a single-level decomposition on high-frequency and multiscale solutions, together with strong and weak scaling [57] (E3).

The evidence therefore supports domain decomposition as a useful scaling strategy, but does not establish a preferred decomposition scheme. Decomposition introduces interface conditions, partitioning choices, and communication or implementation costs. Direct shared-benchmark comparisons among cPINN, XPINN, FBPINN, and multilevel variants remain limited.

8.7 Quantum and hybrid quantum-classical PINNs

Quantum PINNs incorporate parameterized quantum circuits into physics-informed models, either as the primary representation or as part of a hybrid quantum-classical architecture. Fully quantum constructions have been demonstrated for ordinary and partial differential equations [99], while hybrid models have been applied to Navier–Stokes, Maxwell's equations, and nonlinear PDEs [40, 71, 100]. Other approaches explore trainable quantum feature maps [101] or quantum evaluation of nonlocal and fractional operators [51]. The quantum component therefore primarily modifies the representation or computational realization of the physics-informed model.

The central question is whether these representations provide accuracy, parameter-efficiency, or computational advantages over classical PINNs. Early comparisons report mixed results. A systematic study across five benchmark PDEs finds the quantum approach outperforming a classical baseline on two problems, underperforming on two, and performing comparably on the fifth [102] (E3). A parameter-matched comparison likewise reports comparable rather than superior accuracy [101]. A further comparison finds similar performance on several low-dimensional smooth problems, faster and more consistent convergence for one Burgers' case, but difficulty with discontinuous solutions [103] (E3). In a multi-generator power-system problem, entangled

quantum circuits achieve lower physics residuals than separable circuits at comparable parameter budgets, while the advantage over an iso-parameter classical PINN decreases with system size [104] (E3). Neither finding has independent research-group corroboration.

Computational cost remains a major limitation. Quantum approaches have required substantially more computation than classical baselines where directly measured, including approximately four times the wall-clock time in one Poisson problem and a fifteen- to forty-fold increase in per-iteration training time in a broader benchmark [99, 102]. Reported parameter savings do not necessarily imply parameter efficiency under matched budgets. Quantum- specific training difficulties, including loss-landscape collapse in a Maxwell problem, have also been reported [40].

The current evidence is dominated by classical simulation rather than quantum hardware, and comparisons vary in parameter budgets, circuit construction, and training procedures. Noise effects have been examined only in limited settings and depend on the problem [99]. Quantum PINNs are therefore best viewed as an active research direction rather than a general replacement for classical PINNs. Their current evidence grade for a general recommendation remains $\emptyset$.

9 Limitations and Research Agenda

The conclusions of this survey are constrained by the scope and heterogeneity of the available comparative evidence. These limitations and the resulting priorities for future work are summarized in Table 7.

The resulting research priority is not the development of isolated PINN variants, but controlled comparisons that make existing design choices meaningfully distinguishable. Shared benchmarks, independent replication, explicit accuracy–cost reporting, and controlled evaluation of interacting choices would improve the transferability of PINN design guidance.

10 Conclusion

The central challenge in PINN method selection is not the availability of variants, but determining when a particular design choice is appropriate for a given problem. This survey addressed this challenge by organizing the design space into three layers with distinct functional roles: a physics layer containing formulation and closely coupled constraint enforcement; a representation layer containing neural architecture; and a computational-realization layer containing the numerical evaluation of the derivatives and operators required by the formulation. These choices were then related to mathematical, geometric, data, and computational characteristics of the target problem, with comparative evidence assessed according to its strength and independence.

The comparative synthesis does not support a universal PINN configuration. Instead, the evidence supports several conditional recommendations. Hard enforcement is well supported when exact boundary or initial satisfaction is required and an admissible construction exists, although the preferred enforcement mechanism remains problem-dependent in other settings. Energy formulations can be advantageous when

a suitable variational principle is available, particularly where they reduce derivative requirements. Numerical operator and derivative evaluation is also a substantive design choice: multiple studies report competitive or improved accuracy at lower training cost for numerical alternatives to automatic differentiation in the tested settings, although no universal ranking among these alternatives is established. Other choices, including architecture, optimization, sampling, decomposition, and loss weighting, show useful conditional trends but remain sensitive to problem characteristics and experimental configuration.

These findings motivate the problem-driven design procedure proposed in this survey. The procedure begins by characterizing the mathematical, geometric, data, and computational properties of the target problem, identifies structurally admissible choices, consults the available comparative evidence and its grade, accounts for documented interactions, and finally validates the selected configuration on the target problem. The procedure deliberately permits an "insufficient evidence" outcome, so that a design choice is not elevated to a general recommendation solely because it is new, frequently studied, or successful in a particular experimental setting.

The survey also examined emerging directions including KAN/PIKAN, attention-based models, operator learning, adaptive and automated design, data-assisted and uncertainty-aware PINNs, scalable and distributed approaches, and quantum or hybrid quantum-classical PINNs. Their evidence varies substantially: some have multiple comparative studies, whereas others remain supported mainly by demonstrations

Table 7 Limitations identified from the comparative evidence and corresponding directions for future work.

Limitation / research gap	Why it matters	Current limitation	Actionable direction
Scope of the literature corpus	The synthesis reflects the search and eligibility criteria used to identify comparative evidence.	The corpus is curated rather than an exhaustive assessment of all PINN literature.	Extend future reviews with broader searches and updated screening while retaining explicit eligibility and evidence-grading criteria.
Standardized cross-method comparison	Unmatched conditions can confound the effect of a design choice with architecture, parameter budget, optimization, or computational cost.	Parameter count, training budget, seeds, and computational cost are not consistently controlled or reported.	Use shared benchmarks with matched parameter and training budgets, multiple seeds, variance estimates, and accuracy–cost measurements.
Independent corroboration of design choices	Repeated results from one group do not provide independent replication.	KAN/MLP comparisons remain concentrated within one research group [66, 69, 88].	Repeat matched comparisons across independent groups and explicitly relate new findings to prior studies.
Choice among numerical derivative mechanisms	Numerical alternatives to AD show promise, but no preferred alternative is established.	Studies differ in operators, geometries, architectures, and computational budgets, preventing a common ranking (Section 5.5).	Benchmark AD, finite difference, finite volume, RBF-FD, and differential quadrature under fixed formulation, architecture, sampling, and training conditions.
Interactions among design choices	The effect of one choice can depend on another.	Joint comparisons remain limited; evidence includes interactions among training enhancements [74] and between loss weighting and activation [65].	Use factorial or structured ablations that vary multiple design choices while controlling remaining conditions.
Choice of decomposition scheme for scalable PINNs	Decomposition is useful for some large, complex, and multiscale problems, but no preferred scheme is established.	cPINN, XPINN, FBPINN, and multilevel variants lack a common comparison.	Use matched benchmarks reporting subdomain count, interface treatment, communication cost, and scaling behaviour.
Comparative evidence for emerging directions	Demonstrations alone cannot establish general design recommendations.	Transformer, operator-learning, automated-design, and quantum PINNs have limited cross-method or cross-group comparative evidence.	Compare emerging approaches with strong classical PINN baselines under matched problem, parameter, optimization, and computational conditions.

or limited comparisons. Quantum PINNs in particular show feasibility and problem-dependent behaviour, but the current evidence does not establish a general advantage over classical PINNs. These directions therefore remain opportunities for future investigation rather than replacements for the better-supported choices of the core framework.

The main contribution of this survey is therefore not a single preferred PINN variant, but a framework for making design choices conditional on problem characteristics and on the evidence available for each choice. The resulting research agenda is to develop shared-protocol and independently replicated comparisons that report accuracy, computational cost, parameter budget, variance, and implementation details under common problem settings. Such comparisons would allow the field to move from isolated method demonstrations toward more transferable and evidence-based PINN design guidance.

Funding

The authors received no specific funding for this research.

References

[1] Raissi, M., Perdikaris, P., Karniadakis, G.E.: Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics **378**, 686–707 (2019)

[2] Cai, S., Mao, Z., Wang, Z., Yin, M., Karniadakis, G.E.: Physics-informed neural networks (PINNs) for fluid mechanics: A review. Acta Mechanica Sinica **37**(12), 1727–1738 (2021)

[3] Raissi, M., Yazdani, A., Karniadakis, G.E.: Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations. Science **367**(6481), 1026–1030 (2020)

[4] Arzani, A., Wang, J.-X., D'Souza, R.M.: Uncovering near-wall blood flow from sparse data with physics-informed neural networks. Physics of Fluids **33**(7), 071905 (2021)

[5] Cai, S., Wang, Z., Wang, S., Perdikaris, P., Karniadakis, G.E.: Physics-informed neural networks for heat transfer problems. Journal of Heat Transfer **143**(6), 060801 (2021)

[6] Haghighat, E., Raissi, M., Moure, A., Gomez, H., Juanes, R.: A physics-informed deep learning framework for inversion and surrogate modeling in solid mechanics. Computer Methods in Applied Mechanics and Engineering **379**, 113741 (2021)

[7] Manav, M., Molinaro, R., Mishra, S., De Lorenzis, L.: Phase-field modeling of fracture with physics-informed deep learning. Computer Methods in Applied Mechanics and Engineering **429**, 117104 (2024) https://doi.org/10.1016/j.cma.2024.117104

[8] Rao, C., Sun, H., Liu, Y.: Physics-informed deep learning for computational elastodynamics without labeled data. Journal of Engineering Mechanics **147**(8), 04021043 (2021)

[9] Sahli Costabal, F., Yang, Y., Perdikaris, P., Hurtado, D.E., Kuhl, E.: Physics-informed neural networks for cardiac activation mapping. Frontiers in Physics **8**, 42 (2020)

[10] Song, C., Alkhalifah, T.: Solving the frequency-domain acoustic VTI wave equation using physics-informed neural networks. Geophysical Journal International **225**(2), 846–859 (2021)

[11] Chen, Y., Lu, L., Karniadakis, G.E., Dal Negro, L.: Physics-informed neural networks for inverse problems in nano-optics and metamaterials. Optics Express **28**(8), 11618–11633 (2020)

[12] Karniadakis, G.E., Kevrekidis, I.G., Lu, L., Perdikaris, P., Wang, S., Yang, L.: Physics-informed machine learning. Nature Reviews Physics **3**(6), 422–440 (2021)

[13] Cuomo, S., Di Cola, V.S., Giampaolo, F., Rozza, G., Raissi, M., Piccialli, F.: Scientific machine learning through physics-informed neural networks: Where we are and what's next. Journal of Scientific Computing **92**(3), 88 (2022)

[14] Toscano, J.D., Oommen, V., Varghese, A.J., Zou, Z., Ahmadi Daryakenari, N., Wu, C., Karniadakis, G.E.: From PINNs to PIKANs: Recent advances in physics-informed machine learning. Machine Learning for Computational Science and Engineering **1**(1), 1–43 (2025)

[15] Khanra, S., Kukreja, V.K., Bala, I.: Physics-informed neural networks for differential equation solutions: A comprehensive review. Neurocomputing **680**, 133317 (2026) https://doi.org/10.1016/j.neucom.2026.133317

[16] Esmaeilbeigi, M., Annunziata, D., Cuomo, S., Piccialli, F.: Advancements and future directions in loss function designs for physics-informed neural networks: A comprehensive review. Journal of Scientific Computing **109**(1), 32 (2026) https://doi.org/10.1007/s10915-026-03447-w

[17] Azam, M., Chuhan, I.S., Ahmed, M.S., Arshid, K.: A benchmarking study for algorithm selection in scientific machine learning (SciML): PINN vs. gPINN for solving partial differential equations. AppliedMath **6**(2), 26 (2026)

[18] Abbasi, J., Jagtap, A.D., Moseley, B., Hiorth, A., Andersen, P.Ø.: Challenges and advancements in modeling shock fronts with physics-informed neural networks: A review and benchmarking study. Neurocomputing **657**, 131440 (2025)

[19] Lawal, Z.K., Yassin, H., Lai, D.T.C., Che Idris, A.: Physics-informed neural network (PINN) evolution and beyond: A systematic literature review and bibliometric analysis. Big Data and Cognitive Computing **6**(4), 140 (2022)

[20] Gryt, P., Przystałka, P.: Physics-informed neural networks in aerospace engineering: A systematic review of architectures, training strategies, and open challenges. Applied Sciences **16**(13), 6282 (2026) https://doi.org/10.3390/app16136282

[21] Kharazmi, E., Zhang, Z., Karniadakis, G.E.: hp-VPINNs: Variational physics-informed neural networks with domain decomposition. Computer Methods in Applied Mechanics and Engineering **374**, 113547 (2021)

[22] E, W., Yu, B.: The deep Ritz method: A deep learning-based numerical algorithm for solving variational problems. Communications in Mathematics and Statistics **6**(1), 1–12 (2018)

[23] Goswami, S., Anitescu, C., Chakraborty, S., Rabczuk, T.: Transfer learning enhanced physics informed neural network for phase-field modeling of fracture. Theoretical and Applied Fracture Mechanics **106**, 102447 (2020) https://doi.org/10.1016/j.tafmec.2019.102447

[24] Fuhg, J.N., Bouklas, N.: The mixed deep energy method for resolving concentration features in finite strain hyperelasticity. Journal of Computational Physics **451**, 110839 (2022) https://doi.org/10.1016/j.jcp.2021.110839

[25] Wang, S., Teng, Y., Perdikaris, P.: Understanding and mitigating gradient flow pathologies in physics-informed neural networks. SIAM Journal on Scientific Computing **43**(5), 3055–3081 (2021)

[26] Lagaris, I.E., Likas, A., Fotiadis, D.I.: Artificial neural networks for solving ordinary and partial differential equations. IEEE Transactions on Neural Networks **9**(5), 987–1000 (1998)

[27] Sukumar, N., Srivastava, A.: Exact imposition of boundary conditions with distance functions in physics-informed deep neural networks. Computer Methods in Applied Mechanics and Engineering **389**, 114333 (2022)

[28] Lu, L., Pestourie, R., Yao, W., Wang, Z., Verdugo, F., Johnson, S.G.: Physics-informed neural networks with hard constraints for inverse design. SIAM Journal on Scientific Computing **43**(6), 1105–1132 (2021)

[29] Berrone, S., Canuto, C., Pintore, M., Sukumar, N.: Enforcing Dirichlet boundary conditions in physics-informed neural networks and variational physics-informed neural networks. Heliyon **9**(8), 18820 (2023)

[30] Zeinhofer, M., Masri, R., Mardal, K.-A.: A unified framework for the error analysis of physics-informed neural networks. IMA Journal of Numerical Analysis **45**(5), 2988–3025 (2025) https://doi.org/10.1093/imanum/drae081

[31] Chen, Y., Huang, D., Zhang, D., Zeng, J., Wang, N., Zhang, H., Yan, J.: Theory-guided hard constraint projection (HCP): A knowledge-based data-driven scientific machine learning method. Journal of Computational Physics **445**, 110624 (2021) https://doi.org/10.1016/j.jcp.2021.110624

[32] Basir, S., Senocak, I.: Physics and equality constrained artificial neural networks: Application to forward and inverse problems with multi-fidelity data fusion. Journal of Computational Physics **463**, 111301 (2022) https://doi.org/10.1016/j.jcp.2022.111301

[33] Li, X., Deng, J., Wu, J., Zhang, S., Li, W., Wang, Y.-G.: Physical informed neural networks with soft and hard boundary constraints for solving advection-diffusion equations using Fourier expansions. Computers & Mathematics with Applications **159**, 60–75 (2024) https://doi.org/10.1016/j.camwa.2024.01.021

[34] Hao, B., Liu, C., Braga-Neto, U., Wang, L., Zhong, M.: Stability in training PINNs for stiff PDEs: Why initial conditions matter. Foundations of Data Science (2026) https://doi.org/10.3934/fods.2026016 . In press; volume and pages not yet assigned

[35] Alkhadhr, S., Almekkawy, M.: Wave equation modeling via physics-informed neural networks: Models of soft and hard constraints for initial and boundary conditions. Sensors **23**(5), 2792 (2023) https://doi.org/10.3390/s23052792

[36] Lu, L., Jin, P., Pang, G., Zhang, Z., Karniadakis, G.E.: Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. Nature Machine Intelligence **3**(3), 218–229 (2021)

[37] Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., Anandkumar, A.: Fourier neural operator for parametric partial differential equations. In: Proceedings of the International Conference on Learning Representations (ICLR) (2021)

[38] Wang, S., Wang, H., Perdikaris, P.: Learning the solution operator of parametric partial differential equations with physics-informed DeepONets. Science Advances **7**(40), 8605 (2021)

[39] Fang, Z.: A high-efficient hybrid physics-informed neural networks based on convolutional neural network. IEEE Transactions on Neural Networks and Learning

Systems **33**(10), 5514–5526 (2022)

[40] Chen, Z., Shaviner, G.G., Chandravamsi, H., Pisnoy, S., Frankel, S.H., Pereg, U.: Quantum physics-informed neural networks for Maxwell's equations: Circuit design, "black hole" barren plateaus mitigation, and GPU acceleration. Quantum Machine Intelligence **8**(1), 21 (2026) https://doi.org/10.1007/s42484-026-00365-w

[41] Baydin, A.G., Pearlmutter, B.A., Radul, A.A., Siskind, J.M.: Automatic differentiation in machine learning: A survey. Journal of Machine Learning Research **18**(153), 1–43 (2018)

[42] Cho, J., Nam, S., Yang, H., Yun, S.-B., Hong, Y., Park, E.: Separable physics-informed neural networks. In: Advances in Neural Information Processing Systems, vol. 36, pp. 23761–23788 (2023)

[43] Hu, Z., Shi, Z., Karniadakis, G.E., Kawaguchi, K.: Hutchinson trace estimation for high-dimensional and high-order physics-informed neural networks. Computer Methods in Applied Mechanics and Engineering **424**, 116883 (2024) https://doi.org/10.1016/j.cma.2024.116883

[44] Sharma, R., Shankar, V.: Accelerated training of physics-informed neural networks (PINNs) using meshless discretizations. In: Advances in Neural Information Processing Systems, vol. 35, pp. 1034–1046 (2022)

[45] Lim, K.L., Dutta, R., Rotaru, M.: Physics informed neural network using finite difference method. In: 2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC), pp. 1828–1833 (2022). https://doi.org/10.1109/SMC53654.2022.9945171

[46] Xiang, Z., Peng, W., Yao, W., Zhou, W.: Hybrid finite-difference physics-informed neural networks partial differential equation solver for complex geometries. Journal of Thermophysics and Heat Transfer **39**(4), 686–703 (2025) https://doi.org/10.2514/1.T7077

[47] Wei, C., Zhou, Y., Fan, Y., Liu, X., Li, C., Li, X., Wang, H.: Physics-informed neural network based on the finite volume method for solving forward and inverse problems. Physics of Fluids **37**(8), 087190 (2025) https://doi.org/10.1063/5.0284425

[48] Xiao, Y., Yang, L.M., Du, Y.J., Song, Y.X., Shu, C.: Radial basis function-differential quadrature-based physics-informed neural network for steady incompressible flows. Physics of Fluids **35**(7), 073607 (2023) https://doi.org/10.1063/5.0159224

[49] Kassinos, S.C.: Auxiliary finite-difference residual-gradient regularization for PINNs. Machine Learning for Computational Science and Engineering **2**(1), 25

(2026) https://doi.org/10.1007/s44379-026-00076-w

[50] Pang, G., Lu, L., Karniadakis, G.E.: fPINNs: Fractional physics-informed neural networks. SIAM Journal on Scientific Computing **41**(4), 2603–2626 (2019)

[51] Gupta, D., Behera, R.: Quantum physics-informed neural networks for solving integro and fractional PDEs. arXiv preprint arXiv:2606.26865 (2026). Preprint; not peer reviewed

[52] Jagtap, A.D., Kharazmi, E., Karniadakis, G.E.: Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems. Computer Methods in Applied Mechanics and Engineering **365**, 113028 (2020)

[53] Jagtap, A.D., Karniadakis, G.E.: Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations. Communications in Computational Physics **28**(5), 2002–2041 (2020)

[54] Moseley, B., Markham, A., Nissen-Meyer, T.: Finite basis physics-informed neural networks (FBPINNs): A scalable domain decomposition approach for solving differential equations. Advances in Computational Mathematics **49**(4), 62 (2023)

[55] Shukla, K., Jagtap, A.D., Karniadakis, G.E.: Parallel physics-informed neural networks via domain decomposition. Journal of Computational Physics **447**, 110683 (2021)

[56] Hu, Z., Jagtap, A.D., Karniadakis, G.E., Kawaguchi, K.: When do extended physics-informed neural networks (XPINNs) improve generalization? SIAM Journal on Scientific Computing **44**(5), 3158–3182 (2022)

[57] Dolean, V., Heinlein, A., Mishra, S., Moseley, B.: Multilevel domain decomposition-based architectures for physics-informed neural networks. Computer Methods in Applied Mechanics and Engineering **429**, 117116 (2024) https://doi.org/10.1016/j.cma.2024.117116

[58] Lu, L., Meng, X., Mao, Z., Karniadakis, G.E.: DeepXDE: A deep learning library for solving differential equations. SIAM Review **63**(1), 208–228 (2021)

[59] Wu, C., Zhu, M., Tan, Q., Kartha, Y., Lu, L.: A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. Computer Methods in Applied Mechanics and Engineering **403**, 115671 (2023)

[60] Mao, Z., Meng, X.: Physics-informed neural networks with residual/gradient-based adaptive sampling methods for solving partial differential equations with

sharp solutions. Applied Mathematics and Mechanics **44**(7), 1069–1084 (2023) https://doi.org/10.1007/s10483-023-2994-7

[61] Visser, C., Heinlein, A., Giovanardi, B.: PACMANN: Point adaptive collocation method for artificial neural networks. Computer Methods in Applied Mechanics and Engineering **452**, 118723 (2026) https://doi.org/10.1016/j.cma.2025.118723

[62] Wang, S., Yu, X., Perdikaris, P.: When and why PINNs fail to train: A neural tangent kernel perspective. Journal of Computational Physics **449**, 110768 (2022)

[63] McClenny, L.D., Braga-Neto, U.M.: Self-adaptive physics-informed neural networks. Journal of Computational Physics **474**, 111722 (2023)

[64] Bischof, R., Kraus, M.A.: Multi-objective loss balancing for physics-informed deep learning. Computer Methods in Applied Mechanics and Engineering **439**, 117914 (2025) https://doi.org/10.1016/j.cma.2025.117914

[65] Farea, A., Khan, S., Celebi, M.S.: Multi-objective loss balancing in physics-informed neural networks for fluid flow applications. In: 2025 IEEE 32nd International Conference on High Performance Computing, Data, and Analytics (HiPC), pp. 108–118 (2025). https://doi.org/10.1109/HiPC66333.2025.00020

[66] Kiyani, E., Shukla, K., Urbán, J.F., Darbon, J., Karniadakis, G.E.: Optimizing the optimizer for physics-informed neural networks and Kolmogorov–Arnold networks. Computer Methods in Applied Mechanics and Engineering **446**, 118308 (2025) https://doi.org/10.1016/j.cma.2025.118308

[67] Dangel, F., Müller, J., Zeinhofer, M.: Kronecker-factored approximate curvature for physics-informed neural networks. In: Advances in Neural Information Processing Systems, vol. 37, pp. 34582–34636 (2024)

[68] Zhao, Z., Ding, X., Prakash, B.A.: PINNsformer: A transformer-based framework for physics-informed neural networks. In: Proceedings of the International Conference on Learning Representations (ICLR) (2024)

[69] Shukla, K., Toscano, J.D., Wang, Z., Zou, Z., Karniadakis, G.E.: A comprehensive and FAIR comparison between MLP and KAN representations for differential equations and operator networks. Computer Methods in Applied Mechanics and Engineering **431**, 117290 (2024)

[70] Yu, J., Lu, L., Meng, X., Karniadakis, G.E.: Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems. Computer Methods in Applied Mechanics and Engineering **393**, 114823 (2022)

[71] Zabihi, K., Montazeri, H., Suiker, A.S.J.: Hybrid quantum-classical physics-informed neural networks for solving nonlinear PDEs: When and where hybridization is effective? arXiv preprint arXiv:2606.04679 (2026). Preprint; not peer reviewed

[72] Rahaman, N., Baratin, A., Arpit, D., Draxler, F., Lin, M., Hamprecht, F., Bengio, Y., Courville, A.: On the spectral bias of neural networks. In: Proceedings of the International Conference on Machine Learning (ICML). PMLR, vol. 97, pp. 5301–5310 (2019)

[73] Tancik, M., Srinivasan, P., Mildenhall, B., Fridovich-Keil, S., Raghavan, N., Singhal, U., Ramamoorthi, R., Barron, J., Ng, R.: Fourier features let networks learn high frequency functions in low dimensional domains. In: Advances in Neural Information Processing Systems, vol. 33, pp. 7537–7547 (2020)

[74] Yan, W., Zhu, Z., Liu, Z.: GW-RAR-PINN: A novel physics-informed neural network framework for dam-break flow simulation. Engineering Applications of Computational Fluid Mechanics **20**(1), 2679804 (2026)

[75] Ferrer-Sánchez, A., Martín-Guerrero, J.D., Austri, R., Torres-Forné, A., Font, J.A.: Gradient-annihilated PINNs for solving Riemann problems: Application to relativistic hydrodynamics. Computer Methods in Applied Mechanics and Engineering **424**, 116906 (2024) https://doi.org/10.1016/j.cma.2024.116906

[76] Tseng, Y.-H., Lin, T.-S., Hu, W.-F., Lai, M.-C.: A cusp-capturing PINN for elliptic interface problems. Journal of Computational Physics **491**, 112359 (2023) https://doi.org/10.1016/j.jcp.2023.112359

[77] Wight, C.L., Zhao, J.: Solving Allen–Cahn and Cahn–Hilliard equations using the adaptive physics informed neural networks. Communications in Computational Physics **29**(3), 930–954 (2021)

[78] Baniya, S., Maity, D.: A comprehensive review of theoretical concepts and advancements in physics-informed neural networks with applications in structural engineering. Artificial Intelligence Review **59**, 49 (2026)

[79] Đorđević, F., Marinković, M.: PINN surrogate model for nonlinear equilibrium path analysis of von Mises shallow truss. Journal of Big Data **12**, 103 (2025)

[80] Hu, Y., Wu, Y., Li, C., Zhang, D., Chen, J., Yu, A., Zheng, Q.: Reconstruction of the pressure field in dense granular flow using physics-informed neural network. Engineering Applications of Computational Fluid Mechanics **20**(1), 2632247 (2026)

[81] Wang, S., Sankaran, S., Perdikaris, P.: Respecting causality for training physics-informed neural networks. Computer Methods in Applied Mechanics and Engineering **421**, 116813 (2024)

[82] An, G., Xiang, G., Xiang, X., Guedes Soares, C.: Physics informed neural networks based identification modelling of ship maneuvering motion and associated optimal excitation design. Engineering Applications of Computational Fluid Mechanics **19**(1), 2566860 (2025)

[83] Alzubaidi, L., Bai, J., Al-Sabaawi, A., Santamaría, J., Albahri, A.S., Al-dabbagh, B.S.N., Fadhel, M.A., Manoufali, M., Zhang, J., Al-Timemy, A.H., Duan, Y., Abdullah, A., Farhan, L., Lu, Y., Gupta, A., Albu, F., Abbosh, A., Gu, Y.: A survey on deep learning tools dealing with data scarcity: Definitions, challenges, solutions, tips, and applications. Journal of Big Data **10**, 46 (2023)

[84] Mishra, S., Molinaro, R.: Estimates on the generalization error of physics-informed neural networks for approximating a class of inverse problems for PDEs. IMA Journal of Numerical Analysis **42**(2), 981–1022 (2022)

[85] Hao, Z., Yao, J., Su, C., Su, H., Wang, Z., Lu, F., Xia, Z., Zhang, Y., Liu, S., Lu, L., Zhu, J.: PINNacle: A comprehensive benchmark of physics-informed neural networks for solving PDEs. In: Advances in Neural Information Processing Systems, Datasets and Benchmarks Track (2024)

[86] Sun, D., Chen, J., Wang, X., Tang, J.: UniPINN: A unified PINN framework for multi-task learning of diverse Navier–Stokes equations. arXiv preprint arXiv:2603.10466 (2026). Preprint; not peer reviewed

[87] Zou, Z., Meng, X., Karniadakis, G.E.: Uncertainty quantification for noisy inputs–outputs in physics-informed neural networks and neural operators. Computer Methods in Applied Mechanics and Engineering **433**, 117479 (2025) https://doi.org/10.1016/j.cma.2024.117479

[88] Ahmadi Daryakenari, N., Shukla, K., Karniadakis, G.E.: Representation meets optimization: Training PINNs and PIKANs for gray-box discovery in systems pharmacology. Computers in Biology and Medicine **201**, 111393 (2026) https://doi.org/10.1016/j.compbiomed.2025.111393

[89] McGreivy, N., Hakim, A.: Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations. Nature Machine Intelligence **6**(10), 1256–1269 (2024)

[90] Faroughi, S.A., Mostajeran, F.: Neural tangent kernel analysis to probe convergence in physics-informed neural solvers: PIKANs vs. PINNs. Computers & Mathematics with Applications **215**, 155–191 (2026) https://doi.org/10.1016/j.camwa.2026.05.003

[91] Rigas, S., Anagnostopoulos, F., Papachristou, M., Alexandridis, G.: Training deep physics-informed Kolmogorov–Arnold networks. Computer Methods in Applied Mechanics and Engineering **452**, 118761 (2026) https://doi.org/10.1016/j.cma.2026.118761

[92] Santos, F.D., Akhound-Sadegh, T., Ravanbakhsh, S.: Physics-informed transformer networks. In: The Symbiosis of Deep Learning and Differential Equations III (NeurIPS Workshop) (2023). Non-archival workshop paper

[93] Wu, H., Luo, H., Wang, H., Wang, J., Long, M.: Transolver: A fast transformer solver for PDEs on general geometries. In: Proceedings of the International Conference on Machine Learning (ICML) (2024)

[94] Wang, Y., Zhong, L.: NAS-PINN: Neural architecture search-guided physics-informed neural network for solving PDEs. Journal of Computational Physics **496**, 112603 (2024)

[95] Wang, Y., Han, X., Chang, C.-Y., Zha, D., Braga-Neto, U., Hu, X.: Auto-PINN: Understanding and optimizing physics-informed neural architecture. arXiv preprint arXiv:2205.13748 (2022). Preprint; not peer reviewed

[96] Wong, J.C., Gupta, A., Ooi, C.C., Chiu, P.-H., Liu, J., Ong, Y.-S.: Evolutionary optimization of physics-informed neural networks: Evo-PINN frontiers and opportunities. IEEE Computational Intelligence Magazine **21**(1), 16–36 (2026) https://doi.org/10.1109/MCI.2025.3607749

[97] Rui, E.-Z., Chen, Z.-W., Ni, Y.-Q., Yuan, L., Zeng, G.-Z.: Reconstruction of 3D flow field around a building model in wind tunnel: A novel physics-informed neural network framework adopting dynamic prioritization self-adaptive loss balance strategy. Engineering Applications of Computational Fluid Mechanics **17**(1), 2238849 (2023)

[98] Yang, L., Meng, X., Karniadakis, G.E.: B-PINNs: Bayesian physics-informed neural networks for forward and inverse PDE problems with noisy data. Journal of Computational Physics **425**, 109913 (2021) https://doi.org/10.1016/j.jcp.2020.109913

[99] Trahan, C., Loveland, M., Dent, S.: Quantum physics-informed neural networks. Entropy **26**(8), 649 (2024)

[100] Sedykh, A., Podapaka, M., Sagingalieva, A., Pinto, K., Pflitsch, M., Melnikov, A.: Hybrid quantum physics-informed neural networks for simulating computational fluid dynamics in complex shapes. Machine Learning: Science and Technology **5**(2), 025045 (2024)

[101] Berger, S., Hosters, N., Möller, M.: Trainable embedding quantum physics informed neural networks for solving nonlinear PDEs. Scientific Reports **15**(1), 18823 (2025) https://doi.org/10.1038/s41598-025-02959-z

[102] Farea, A., Khan, S., Celebi, M.S.: QCPINN: Quantum-classical physics-informed neural networks for solving PDEs. Machine Learning: Science and Technology **6**(4), 045053 (2025) https://doi.org/10.1088/2632-2153/ae1c91

[103] Siegl, P., Wassing, S., Mieth, D.M., Langer, S., Bekemeyer, P.: Solving transport equations on quantum computers: Potential and limitations of physics-informed quantum circuits. CEAS Aeronautical Journal **16**(1), 63–80 (2025) https://doi.org/10.1007/s13272-024-00774-2

[104] Kavisankar, L., Das, S., Mishra, P.: Entanglement as a structural prior: Comparative analysis of separable and entangled quantum physics-informed neural networks for coupled power system dynamics. In: 2026 International Conference on Intelligent and Sustainable AI Systems (ICOSAAS), pp. 692–698 (2026). https://doi.org/10.1109/ICOSAAS68663.2026.11648767